From Self-Improvement to Recursive Self-Improvement in Physical AI: A Survey

Junzhou Chen[1], **Kaiwen Guo**[2], **Tianyu Shi**[3], **Lingjuan Lv**[2]
[1]William & Mary, [2]Sony AI, [3]McGill University

Abstract

Physical AI increasingly learns from interaction after deployment, raising the question of how robots can improve both their capabilities and their learning processes. This survey reviews mechanisms for acquiring experience, constructing feedback, and retaining updates to policies, memories, predictive models, and learning code. We compare autonomous practice, reward-based and hindsight-goal supervision, and update methods ranging from local adaptation to generalist retraining. We distinguish physical self-improvement, which retains capability gains in an embodied system, from recursive self-improvement, which uses earlier improvement outcomes to learn a better process for producing subsequent gains. These learning choices interact through recovery requirements, hardware condition, human effort, and risk. We review datasets, simulation and collection tools, and evaluation methods, identifying gaps in assessing improvement processes. We propose comparing revised and frozen predecessor processes through subsequent physical learning under matched starting conditions and resources. A key open question is which choices of experience, feedback, and update strategy lead to better subsequent learning. We outline a research roadmap for controllers that learn these choices within defined safety interfaces, progressing toward the transfer of improvement strategies across tasks and embodiments. We provide a regularly updated project page to document ongoing progress: `https://github.com/Junzhou-Chen/Awesome-Physical-RSI`

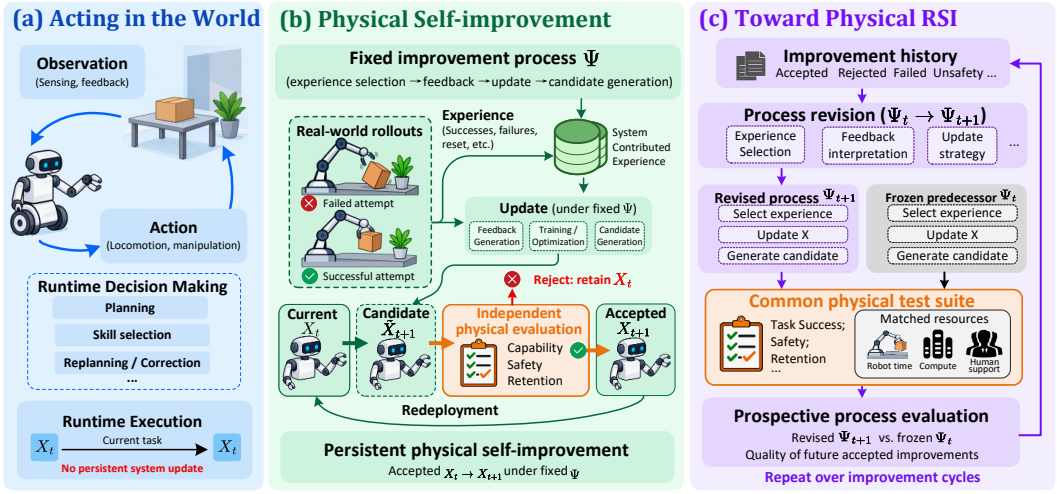


Figure 1: (a) Runtime execution produces no persistent system update. (b) Physical SI retains an accepted X update under fixed Ψ. (c) Toward RSI, improvement history guides Ψ revision, evaluated through later physical learning against the frozen predecessor under matched resources.

Preprint.

Contents

1 Introduction **3**

2 Recursive Self-Improvement: Foundations and Requirements **4**

2.1 From Self-Improvement to Recursive Self-Improvement 4

2.2 The Recursive Improvement Loop 5

2.3 Criteria for Recursive Self-Improvement 6

3 Recursive Self-Improvement under Physical Embodiment **7**

3.1 Targets of Physical Improvement 7

3.2 Agentic Inference and Embodied Policy Execution 8

3.3 The Physical Improvement Loop 8

3.4 What Physical Embodiment Changes 10

4 Mechanisms and Systems for Physical Improvement **10**

4.1 Self-Generated Physical Experience 10

4.2 Self-Evaluation and Verification 13

4.3 Persistent Self-Modification 15

4.4 Closing the Physical Improvement Loop 16

4.5 Process-Level Candidates for Physical RSI 16

5 Resources and Evaluation for Physical Improvement **18**

5.1 Resource Capabilities and Experimental Scope 18

5.2 Current Evaluation Targets and Coverage Gaps 20

5.3 A Prospective Comparison of Improvement Processes 21

5.3.1 Improvement Trajectories as the Benchmark Unit 22

6 Opportunities and Challenges of Physical RSI **23**

6.1 Opportunities 23

6.2 Technical and Physical Challenges 24

6.3 Safety and Trustworthiness 26

7 Toward Recursive Self-Improving Physical AI **27**

7.1 Missing Capabilities for Physical RSI 27

7.2 Roadmap to Physical RSI 29

8 Conclusion **30**

1 Introduction

Physical AI increasingly learns through deployment. A robot can collect experience, train a better policy, and return to practice; generalist retraining, autonomous instruction-following practice, and learning with feedback from a frozen earlier checkpoint already demonstrate useful forms of this cycle [19, 212, 147]. The next question concerns the organization of learning: *can the outcomes of earlier updates teach a robot to choose more effective ways of improving itself?* We distinguish physical self-improvement (SI), which produces a persistent capability gain, from recursive self-improvement (RSI), which learns a better process for producing subsequent gains. This distinction directs attention to the decisions linking one deployed version to the next: what to practice, which feedback to use, and what to change. Figure 1 depicts runtime execution, retained system updates, and process revisions.

Several technical developments make this question timely. Generalist action models provide reusable starting policies, and heterogeneous robot datasets broaden their behavioral coverage [21, 22, 160, 87, 40, 167, 86]. On-robot reinforcement learning, navigation fine-tuning, and intervention-assisted manipulation adapt these priors or task-specific learners to consequential physical experience [85, 121, 67, 110, 91]. The resulting update paths differ in their dependence on demonstrations, rewards, human correction, and recovery. Choosing among them becomes a learning problem when the same robot must continue improving across new tasks and changing deployment conditions.

Physical practice couples these choices. A proposed task occupies hardware and changes the scene. Its outcome becomes useful training data only if it can be observed and the robot can continue afterward. Learned undo behavior sustains repeated practice, fleet orchestration allocates tasks, and asynchronous collection connects those tasks to successive policies [149, 8, 198]. Analyses of autonomous collection show that preparation, reset, and outcome labeling remain substantial costs [123]. Success detection, damage assessment, and runtime monitoring capture different consequences of an experiment [44, 15, 6]. Virtual practice offers cheaper search through visual and dynamics variation, but its usefulness depends on how well it predicts behavior on hardware [162, 133, 97]. Thus the value of an experience depends jointly on the feedback it provides, the update it can support, and the practice opportunities it leaves available.

Existing surveys organize the enabling literature from complementary perspectives. Reviews of self-evolving agents classify update targets, timing, and mechanisms; world-model and vision–language–action (VLA) reviews explain predictive representations, model architectures, and action interfaces [53, 42, 115]. Recent preprints extend this coverage to VLA data engines, foundation-model methods in robotics, and connections among representations, policies, and world models [176, 134, 120]. These perspectives clarify what can change within a robot-learning system. Our focus is the dependency between those changes: how the chosen experience supports a particular repair, and how the outcome of that repair informs later learning decisions.

Directly related SI and RSI preprint surveys provide three further organizing views. An update-centered account connects modification targets, loop closure, and evaluator hierarchies [28]. A model–scaffold account formalizes self-induced committed changes and their driving signals [140]. An autonomy-centered roadmap distinguishes increasingly system-directed roles across execution, strategy, experience, environment, and meta-improvement [46]. We connect these views to the physical conditions under which improvement occurs: recoverable trials, observable outcomes, retained capabilities, and the human effort needed to sustain learning. Complementary benchmarks expose skill retention, everyday-activity coverage, and constrained-learning behavior [105, 95, 79]. Assessing an improvement process requires connecting such component measurements to the learning it produces over time.

There are established precedents for making learning decisions trainable. Meta-learning and learned optimizers adapt update behavior across problems; algorithm and workflow search make learning procedures explicit search targets [48, 9, 137, 71]. Agentic revision of robot-learning code brings part of this edit space to physical experiments, although current evidence remains scoped to the reported tasks and interfaces [181]. Generated tasks and reward programs broaden inexpensive candidate search in simulation [175, 114]. Together these mechanisms motivate a bounded improvement controller that learns from the downstream effects of its experiment, feedback, and update choices. Such a controller can operate over reusable execution, recovery, and safety interfaces while retaining a clear physical learning target.

This problem-driven narrative survey covers manipulation, navigation, and locomotion through their contributions to persistent learning and process adaptation. Studies were selected through topic-driven searches and citation tracing, with targeted source checks completed on 10 October 2026. We prioritize original research and identify preprints explicitly. Comparisons track tasks and robots, experience sources, retained changes, learning decisions, evaluation, and human contributions; unavailable details remain unspecified.

The survey makes three contributions. First, it develops a two-level SI–RSI framework that separates retained system gains from learned improvement decisions. Second, it compares the mechanisms linking physical experience, feedback, and persistent updates, explaining how their dependencies shape learning efficiency and continued practice. Third, it proposes a prospective comparison of revised and predecessor processes and a research roadmap centered on credit assignment to improvement decisions. Sections 2 and 3 establish the concepts and physical interfaces; Section 4 examines mechanisms and systems; Section 5 develops resources and evaluation. The final sections analyze coupled bottlenecks and prioritize the capabilities an improvement controller should learn.

2 Recursive Self-Improvement: Foundations and Requirements

Learning can change an embodied system's behavior, its retained capabilities, or the procedure used to acquire those capabilities. These changes can occur together, but they answer different questions. Adaptation concerns a response to new conditions; continual learning concerns accumulation and retention; SI concerns a system's contribution to a persistent gain; RSI concerns learning more effective improvement decisions. Update-target, timing, and mechanism taxonomies provide useful background for locating these changes [53]. We use two functional objects, the system state X and its improvement process Ψ, to connect them across learning cycles.

2.1 From Self-Improvement to Recursive Self-Improvement

Adaptation changes behavior or internal state in response to a task or environment. Meta-learning prepares this response across task distributions: model-agnostic meta-learning trains an initialization for rapid parameter adaptation, while robot meta-imitation studies one-shot visual skill acquisition in simulation and on hardware [48, 49]. Learned optimizers place trainable structure in the update rule itself. An LSTM can produce parameter updates for related tasks, and a recurrent optimizer can learn derivative-free search [9, 32]. These mechanisms establish that update behavior can be learned. Whether that behavior continues to improve during deployment depends on what feedback reaches the update rule and which of its decisions remain adaptive. Reinforcement learning (RL) specifies an optimization approach; its use can be researcher-directed, automated within a fixed pipeline, or selected as one intervention among others.

Continual and lifelong learning focus on capability accumulation over sequences of tasks. Elastic weight consolidation restricts changes to parameters important for earlier tasks; gradient episodic memory constrains updates with stored examples; replay revisits past experience [88, 107, 141]. Robot-specific studies extend retention to language-guided skill acquisition and to vision–language capabilities during action training [34, 61]. These approaches concern where plasticity is allowed and which capabilities survive later learning. They therefore complement SI and process learning: an effective improvement decision must account for both the new gain and its effect on existing behavior.

Curriculum learning makes the selection and ordering of experience explicit [17]. Automatic goal generation targets an appropriate difficulty frontier; asymmetric self-play produces challenges through interaction; multi-agent autocurricula generate evolving demands, including emergent tool use [50, 159, 14]. Regret-based environment design generates solvable challenges from a protagonist–antagonist performance gap, while prioritized level replay revisits levels with estimated future learning potential [41, 82]. Difficulty, regret, and learning potential define different selection objectives. This distinction matters for process learning because a curriculum can respond to a changing learner under a fixed selection rule, or the selection rule can itself learn from the subsequent value of earlier choices.

Persistence determines what an interaction contributes to later tasks. Self-feedback and tool-grounded critique can refine the current answer; retained verbal reflections and executable skills instead carry information across attempts [116, 56, 152, 170]. Executable-skill accumulation in Minecraft illustrates this distinction in a digital environment. Physical reuse additionally requires checking that

the skill or memory remains valid on the robot and in the changed scene. The retained object and its lifetime are therefore more informative than the number of reasoning or correction steps.

Let X_t denote the persistent state of a system at improvement cycle t. Let Ψ_t denote the process that selects experience, constructs or interprets feedback, chooses a modification, and decides whether to retain it. With experience and feedback H_t, the cycle can be written schematically as

$$X_{t+1} = \Psi_t(X_t; H_t). \tag{1}$$

The expression includes proposal and acceptance: an unsuccessful candidate can leave the retained system unchanged. X and Ψ describe functional roles, so one model may act, evaluate outcomes, and propose changes. A model–scaffold decomposition offers a complementary view of the software objects that implement these roles [140]. We identify a process revision through a persistent change to the decision rule controlling experience, feedback, modification, or acceptance. New observations, a larger candidate pool, or different outputs from an unchanged rule can influence learning without changing that rule.

In this survey, physical SI requires a capability gain that persists beyond the current interaction, is assessed on a specified external objective, and is supported by a traceable system contribution to its experience, proposal, or feedback. Human-defined goals, demonstrations, and fixed safety constraints remain compatible with this scope; their roles determine how much of the improvement is system-directed. RSI adds a learned process revision whose effect is assessed through subsequent learning. This definition separates the origin of a proposed change, its persistence, and its demonstrated benefit.

Repeated SI can occur under an unchanged process:

$$X_0 \longrightarrow X_1 \longrightarrow X_2 \longrightarrow \cdots. \tag{2}$$

Bootstrapping from self-generated rationales and distilling self-generated mistakes supply training signal within researcher-specified update objectives [200, 2]. Self-generated update directives similarly change the material entering a later training step [214]. For process learning, the additional target is the rule governing such choices. Digital search exposes several candidate targets: proposing candidates from earlier scores, evolving task and mutation prompts, constructing algorithms from primitive operations, and revising workflows or code [189, 47, 137, 71, 201]. These mechanisms broaden the space of learnable improvement decisions; physical RSI connects their revisions to later gains obtained through robot interaction.

2.2 The Recursive Improvement Loop

An improvement cycle has five functions: acquire experience or propose changes; derive learning feedback; modify a retained object; validate and accept or reject the candidate; and reuse the accepted state. Learning feedback ranks or trains candidates, while validation measures the resulting behavior. Human comparisons and model-generated rewards differ in their information sources and supervision costs [37, 197]. Their value depends on how well the signal guides learning and how the resulting gain is measured.

Reward research explains this separation. Inverse reward design treats a specified reward as evidence of the intended objective; reward-gaming analysis characterizes divergence between proxy and true return; proxy-model overoptimization experiments show a related effect in synthetic preference learning [60, 153, 55]. A stable outcome measure allows a learning signal to be adapted while preserving a comparison of the behavior it induces. The detailed physical protocol is developed in Section 5.3.

Physical systems implement the five functions with different dependencies. MEDAL++ learns task and undo behaviors using a visual reward derived from initial demonstrations, supporting continued practice [149]. Mobile manipulation combines task-relevant exploration, behavior priors, and semantic with low-level rewards; visual navigation can combine offline model-based pretraining with online model-free fine-tuning [121, 67]. Generalist retraining reuses earlier agent trajectories, foundation-model-guided collection supplies hindsight goals for a goal-conditioned policy, and frozen-checkpoint feedback supplies a practice signal for later policy learning [19, 212, 147]. These routes address recovery, exploration, supervision, and data reuse at different points in the loop.

Simulation exposes additional proposal interfaces. Generated tasks and scenes, language-generated training tasks, and searched reward programs change what a learner practices or optimizes [175, 172,

114]. They support inexpensive candidate search; paired hardware outcomes determine when those candidates are useful for a physical learner.

The outer loop uses the consequences of earlier updates to revise the next learning decision. Schematically,

$$\Psi_{t+1} = \mathcal{M}(\Psi_t; H_{\leq t+1}).$$

At this level, the meta-updater $\mathcal{M}$ can remain fixed while Ψ learns. For example, a task selector may learn which diagnosed weakness benefits from contact-focused practice and which requires a model update. The feedback for this choice arrives after the selected intervention has changed the learner. Accepted gains, failed repairs, and their costs provide complementary observations of intervention value.

We adopt a bounded empirical account of RSI within a task family, hardware configuration, and resource envelope. Contemporary preprint perspectives distinguish bounded refinement from open-ended recursion and organize increasingly autonomous improvement roles [28, 46]. Historically, Gödel machines also make the proof-searching program editable, with utility proofs governing rewrites under encoded axioms [143]. Physical process learning instead relies on empirical comparisons under uncertain dynamics. Visual verification illustrates an interface that can support immediate action screening and later policy improvement [203]; a process learner can use accumulated outcomes to decide when this feedback is informative and when another observation or external check is worth its cost.

Three-stage grasping example. A robot first replans a failed grasp and moves a cup: the current execution improves. It then trains on its trials and retains a better grasping policy: X improves. Finally, earlier update outcomes teach it to select contact-focused practice for one failure class and scene-model revision for another: Ψ changes. A comparison with its saved predecessor on later cup instances tests the future learning value of this diagnosis-to-intervention rule. The example is hypothetical and illustrates the three roles rather than reporting an experiment.

2.3 Criteria for Recursive Self-Improvement

Table 1 summarizes the learning claims and their operating scope. Persistence concerns the system gain; meta-change identifies the learned decision rule; future efficacy concerns the later learning it produces. Self-direction and loop closure describe who operates the cycle and which interfaces run together.

Table 1: Criteria for persistent SI, repeated closed-loop SI, and process-level improvement.

Criterion	Observable question	Role
Persistent gain	Is the accepted change retained and better than X_t on an independent task measure?	SI basis
Self-direction	Who proposes experience and changes, evaluates outcomes, and selects updates? What does a human still do?	Scope
Loop closure	Which steps from proposal to validation and reuse run across successive cycles without per-cycle intervention?	Scope
Meta-change	Which persistent decision rule in Ψ_t changes from the system's improvement history?	RSI basis
Future efficacy	From comparable starting X and history, does revised Ψ_{t+1} improve later retained capability over frozen Ψ_t under common opportunities and budgets?	RSI basis
Multi-generation record	Are successive X_t and Ψ_t, rejected proposals, costs, and outcomes linked and reproducible?	Evidence

Autonomy and learning progress remain separate dimensions. Goal specification, initial setup, safety vetoes, teaching, and experiment selection distribute responsibility in different ways. Identifying these roles makes intervention-assisted and more autonomous systems comparable without obscuring their distinct mechanisms.

Future-process evaluation connects the rule change to retained outcomes. Held-out procedural testing offers a useful generalization design [38]; reset and outcome-detection analyses show why physical comparison must include more than training-trial counts [123]. Statistical self-modification results

provide assumption-specific guidance about permitted changes [169]. Section 5.3 develops these comparisons. The next section examines the physical interfaces that supply their experience and constrain continued learning.

3 Recursive Self-Improvement under Physical Embodiment

A physical improvement cycle must preserve both a useful learner and the conditions for its next experiment. Actions move objects, occupy hardware, and alter what can be observed or attempted afterward. The distinction between system state X and improvement process Ψ therefore has a concrete interface question: which part of the robot-learning system should change, and which physical dependencies must remain reliable while it changes? This section locates repair targets and traces the path from interaction to retained deployment. The mechanisms used to implement these updates are compared in Section 4.

3.1 Targets of Physical Improvement

The retained state of an embodied system extends beyond policy weights. Perception, scene memory, dynamics predictions, success estimates, reusable skills, and control configuration can each limit behavior. These objects carry different information: parameters encode a reusable response, scene memory records local facts, and a self-model describes the machine through which the response is executed. Table 2 organizes them by the physical weakness an update is intended to repair.

Table 2: Physical repair targets and their retained information. Examples identify object types; their update mechanisms and evidence are compared in Section 4. Digital memory examples and simulated morphology search are identified separately from physical deployment.

Repair target	Retained information and examples	Physical weakness addressed
Perception and action model	Visual features and task-conditioned action parameters [21, 22, 160, 87, 36, 199, 125]	Missing behavior or a representation that fails to expose relevant scene information
Memory and skills	Interaction-derived scene state; verbal memories and executable skills in digital agents [81, 152, 170]	Missing local knowledge or behavior that should be reused across attempts
World and self-model	Scene-transition predictions and software estimates of body dynamics [179, 62, 18, 39, 89]	Poor prediction of action effects or changed physical dynamics
Reward or verifier	Success labels, failure diagnoses, and predicted-outcome rankings [44, 45, 180]	Ambiguous progress or incorrect interpretation of an outcome
Improvement harness	Task orchestration, collection–training synchronization, or editable learning code [8, 198, 181]	Practice allocated to the wrong bottleneck or an ineffective update path
Control and embodiment	Controller settings; morphology and controller search in simulation [23, 158, 144]	An interface or body configuration that restricts exploration and achievable control

Behavioral weakness and update scope. Broad retraining and task-local learning change the action-generating state at different scopes. Earlier agent trajectories, foundation-model-guided practice, and frozen-checkpoint feedback can support a later generalist or action policy, whereas deployed manipulation and navigation learning target local conditions [19, 212, 147, 121, 67]. The repair should match the weakness: local data may address a recurring deployment error, while consolidation into a shared policy also affects its earlier skills. Sequential robot-task evaluation makes this retention cost observable [105]. The central interface is the lifetime and reach of the changed behavior.

Temporary conditioning and retained learning occupy different points in this interface. In-context adaptation changes what a pretrained VLA does with the current prompt; few-shot object-centric adaptation changes how it responds to a new object; corrective supervision uses the difference between a proposed action and a human correction [156, 96, 98]. Action-predictive reasoning instead changes representations used by the policy [52]. Comparing these interventions requires identifying what survives beyond the current context and what supervision produced it. This determines whether a successful response can be relied on during the next deployment.

Local knowledge and predictive state. Scene memory is appropriate when the missing information concerns the current workspace. An action-conditioned scene graph retains discoveries made through exploration [81]; its validity can change when objects move, surfaces change, or an interruption alters the scene. A memory interface therefore needs both retrieval and refresh. Predictive models serve a broader role: imagined rollouts and predicted intermediate states can supply search or supervision for policy updates [191, 211]. Their physical value depends on whether the modeled consequences are informative about the proposed repair.

Model transfer and adaptation expose this dependence. Simulation-pretrained world models and model-supported diffusion-policy fine-tuning allocate some learning to virtual prediction, while online dynamics adaptation for off-road driving uses an adaptation basis learned before deployment [94, 26, 93]. Virtual variation can also support a dexterous-catching policy subsequently tested on hardware [138]. The interface to check is the prediction the learner actually uses: a model that produces plausible trajectories may still misrepresent contact or recovery states that determine the value of an update.

Feedback and process state. Retained feedback models affect which outcomes become training signal. Progress estimation from trajectory comparisons, feature elicitation through targeted human explanations, and human-aligned offline navigation preferences provide different information about intended behavior [99, 122, 145]. A feedback error can therefore call for repairing the evaluator rather than collecting more data for the policy. Reward-design and reward-gaming analyses explain why this choice must be connected to an external outcome measure [60, 153].

The improvement harness uses such information to organize later experiments. Information-theoretic exploration, interactive requests for skills or help, and cost-aware acquisition value different observations; retained human feedback can also inform later robot programs [196, 165, 132, 101]. Generated tasks and searched reward programs broaden the proposal space in simulation [175, 114]. Here the X–Ψ boundary is functional: an evaluator update can repair current predictions, while a learned rule for choosing when and how to use that evaluator changes future learning decisions. The latter is tested through the updates it subsequently produces.

3.2 Agentic Inference and Embodied Policy Execution

High-level inference interfaces help locate a failure before a repair is selected. Feasibility-weighted skill selection chooses among available behaviors; generated control programs call perception and action primitives; spatial value maps synthesize end-effector trajectories; feedback-conditioned planning revises the next task step [7, 100, 73, 72]. These interfaces expose different error sources. A skill-selection failure concerns which behavior was invoked, whereas an unsuccessful action primitive concerns how that behavior was executed.

Prospective screening and runtime monitoring provide another view of the failure. Predicted action consequences and visual verification can reject unsuitable proposals; task-agnostic monitoring can trigger a fallback during execution [180, 203, 6]. If useful actions are already available, screening or routing may address the immediate weakness. If every proposal fails, new skill data or a policy update may be needed. Retaining a reliable composition or routing configuration also carries experience across episodes without requiring a full weight update.

Execution traces become learning resources when they retain the observed weakness, intervention, and outcome. Visual verification illustrates the connection by supporting immediate steering and later policy fine-tuning [203]. Task orchestration and agentic experiment revision place related decisions at the practice and training interfaces [8, 181]. The record linking them should identify whether a new behavior was retained or the rule for initiating future learning changed. Section 5.3 gives the corresponding system and process comparisons.

3.3 The Physical Improvement Loop

The physical loop has five interfaces: interaction, logging, verification, persistent modification, and guarded redeployment. Each passes information or a usable physical state to the next. An incomplete reset changes the next trajectory; ambiguous labels change the training objective; a trained checkpoint affects future practice only after deployment. Figure 2 depicts these dependencies and the separate revision of the learning process.

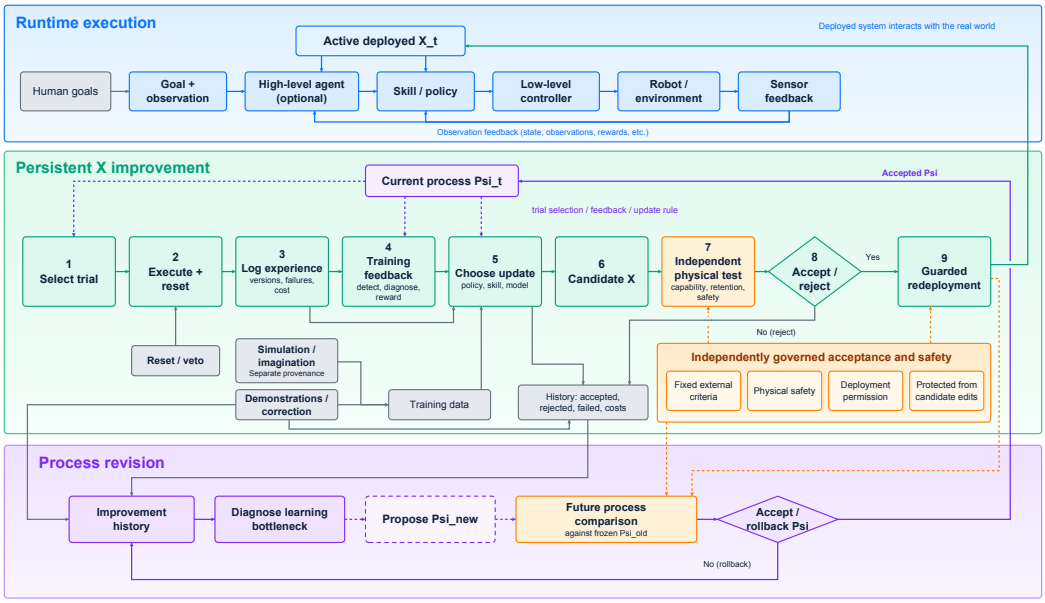


Figure 2: **Embodied execution and two levels of improvement.** Runtime feedback acts through the deployed system. A persistent X update connects logged experience, learning feedback, independent physical acceptance, and redeployment. Historical outcomes can induce a Ψ revision, whose downstream value is assessed through later learning. Rejected candidates remain linked to their supporting trials and costs.

Interaction and continuation. Collection arrangements determine which weaknesses a learner can encounter. Distributed data collection broadens physical coverage; bimanual teaching and portable demonstration interfaces make particular behaviors easier to acquire [86, 209, 35]. On-robot manipulation and walking RL obtain consequential trials, while task-and-undo learning supplies a continuation mechanism [85, 59, 149]. The relevant experience source is thus an arrangement of policy, task, recovery, and supervision. A trial is available for repeated learning only while the arrangement can restore or continue from its terminal state.

Logging and synchronization. A trajectory needs a collecting policy, robot identity, task conditions, outcome, and reset context to be interpretable by a later learner. Asynchronous infrastructure links collection, buffering, training, and policy synchronization [198]. Preparation, reset, and success detection remain material inputs even when action execution is autonomous [123]. Linking those inputs to the deployed version makes it possible to distinguish a useful local failure from data gathered under a different scene or learner.

Verification and diagnosis. Outcome labels answer whether a task succeeded; failure diagnoses identify a possible cause; predicted consequences help decide whether to attempt an action [44, 45, 180]. Human comparisons add judgment at annotation cost [37]. These signals enter different decisions and may disagree: a visually successful placement can still cause damage, and a low progress score may reflect an inadequate camera view. Damage-aware assessment and runtime protection supply consequence-specific observations [15, 6]. Separating learning feedback from acceptance measurements prevents optimization against a changing proxy from obscuring the resulting physical behavior [55].

Modification and redeployment. Selected experience is incorporated into a versioned policy, memory, model, or other retained object. The candidate is then assessed for capability, retention, and relevant safety outcomes before becoming the operating system. Deployment connects the accepted object to the next physical trial; rejection leaves the previous system available. This interface also preserves the distinction between a virtual proposal, an offline-trained candidate, and a version actually used on hardware. The detailed update routes are compared in Section 4.

The outer loop uses the outcomes of these interfaces to revise how later learning is organized. Agentic revision of training code over task-specific reset, safety, and verification services exposes one such edit space [181]. Other possible outputs are a learned task-selection rule or a diagnosis-to-repair

mapping. Their value depends on the future updates they induce, rather than only the policy produced by the triggering experiment.

3.4 What Physical Embodiment Changes

Experiments leave material state. Objects, calibration, and hardware condition persist between trials. Updating a software body model or choosing a compensatory controller can address changed dynamics, while a physical reconfiguration changes the machine that must be tested [18, 39, 89, 23]. Returning to an earlier checkpoint restores software behavior but leaves wear and scene changes in place. The next experiment therefore depends on both the retained learner and the physical state inherited from previous attempts.

Useful feedback is only partially observed. Task completion, contact, damage, and recoverability need different observations. A visual label can be useful for policy learning while omitting force or delayed harm. The choice of an experiment must account for whether its outcome can distinguish plausible repairs and whether further observation or human judgment can resolve the uncertainty.

Practice has coupled costs. Robot time, computation, teaching, preparation, and recovery constrain different parts of the loop. Spending less on one interface can increase demand on another; virtual search can save hardware trials or nominate misleading candidates. An improvement decision is useful when its eventual gain justifies these costs and leaves continued practice feasible. Section 6.2 examines the resulting bottlenecks, while Section 6.3 develops the boundaries on exploration, deployment, and process editing.

4 Mechanisms and Systems for Physical Improvement

Figure 3 organizes the literature by its contribution to the improvement loop.

Section 3 identified the objects and interfaces of physical improvement. Four questions organize the technical comparison here: where experience comes from, how learning signals are constructed, which component retains the change, and what sustains the next deployment. These dimensions reveal why broadly reusable policies, local adaptation, and editable training procedures require different designs for supervision, recovery, and computation.

4.1 Self-Generated Physical Experience

Physical learning draws on both reusable datasets and experience acquired during deployment. RoboNet, DROID, and AgiBot World broaden multi-robot and multi-task coverage; the AgiBot World preprint includes human-in-the-loop verification [40, 86, 24]. ALOHA, UMI, and teleoperated play offer different demonstration interfaces and degrees of task segmentation [209, 35, 111]. RT-1, Octo, and OpenVLA turn broad experience into adaptable starting policies [21, 160, 87]. These resources reduce the skill-acquisition burden at the start of deployment. Subsequent practice has a different purpose: to resolve weaknesses of the current policy and operating environment. A useful data strategy combines broad priors with targeted local experience.

Making physical practice repeatable. QT-Opt and real-world legged RL establish the value of learning from consequential trials [85, 59]. MEDAL++ addresses the complementary problem of returning the robot to a useful training distribution [149]. Initial forward and backward demonstrations seed a task policy and an undo policy; alternating their execution lets the undo behavior return the system toward demonstration states. A visual reward classifier uses final demonstration states as positives and newly collected states as negatives. Image augmentation, demonstration oversampling, and behavior-cloning regularization stabilize policy learning from this small initial dataset. Experiments cover four manipulation tasks on a Franka Panda. The design makes recovery a learned part of the practice apparatus, although demonstration collection, task preparation, and occasional manual resets still contribute to its operation.

HIL-SERL uses correction during a trial to make difficult exploration productive [110]. The robot actor sends transitions to a learner, which combines demonstration and online replay in equal proportions and periodically returns new policy parameters. A human can temporarily take control through teleoperation; those corrective transitions enter both replay buffers, giving them greater influence than autonomous transitions stored only in the online buffer. An offline-trained visual success classifier

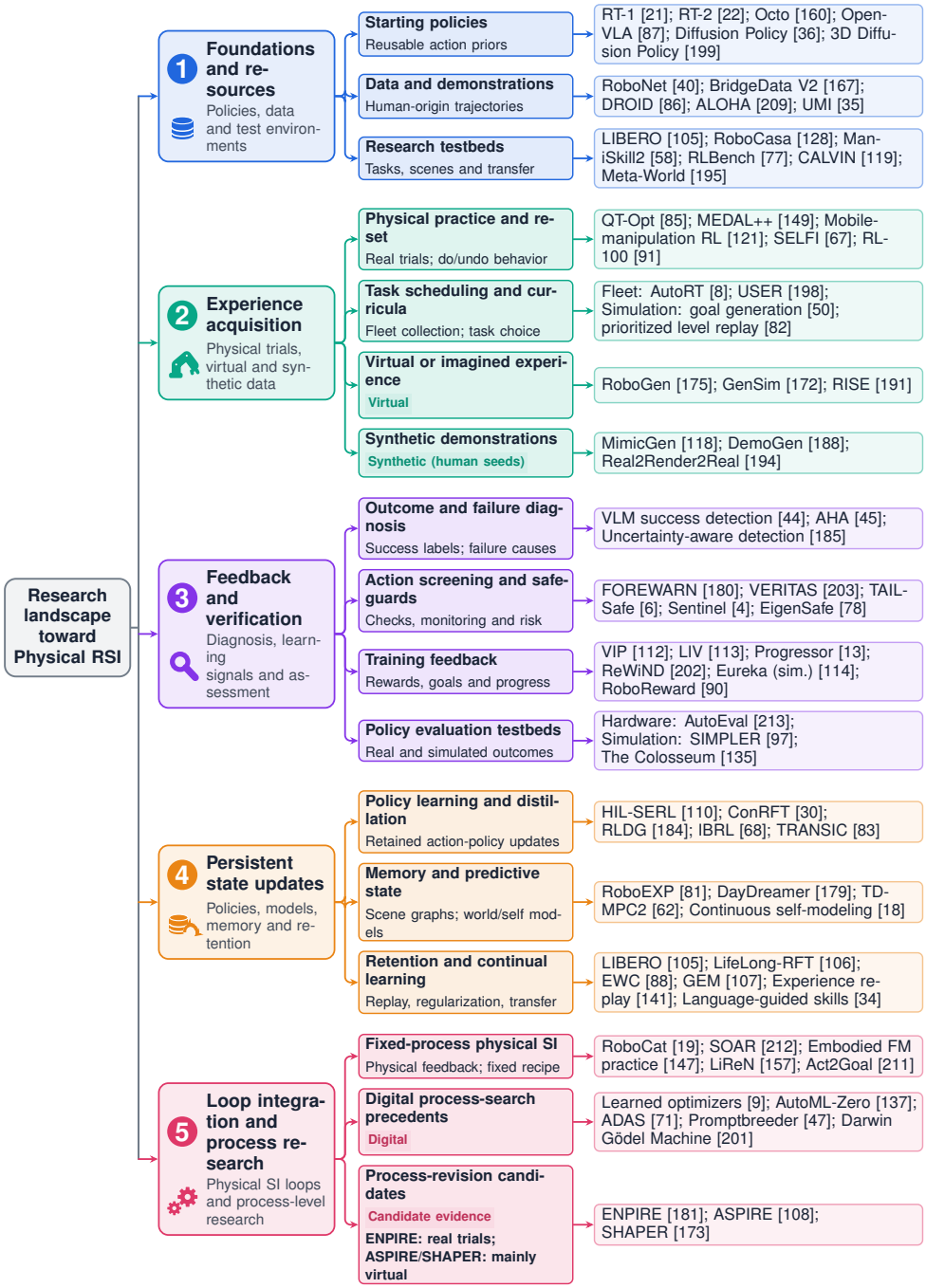


Figure 3: Research landscape toward physical RSI, organized by foundations, experience acquisition, feedback, persistent updates, and loop integration. Representative works connect physical, simulated, synthetic, and digital routes across these functions.

supplies sparse rewards. Precision, dynamic, and bimanual manipulation experiments show how this combination can learn behaviors that are difficult to acquire through unguided physical exploration. The intervention budget and the reset procedure therefore belong alongside success rate and training time in the comparison. SERL supplies reusable infrastructure for this class of real-robot learning [109].

Other routes reduce exploration cost through priors. Mobile-manipulation RL combines task-relevant exploration, behavior priors, and semantic and low-level feedback; SELFI initializes navigation with model-based learning before online model-free adaptation [121, 67]. RL-100 further studies performant real-robot policy learning [91]. Across these designs, recovery preserves access to useful starting states, demonstrations supply actions at difficult states, and prior models reduce the trials needed to discover competent behavior. Their suitability depends on whether the limiting cost is failed exploration, task preparation, or learning from each interaction.

The transition from demonstrations to on-robot practice also depends on where prior knowledge enters learning. Imitation-bootstrapped RL seeds exploration with demonstrated behavior, while Q2RL extracts value estimates from behavior cloning [68, 43]. RLPD repeatedly mixes prior and online trajectories, preserving demonstrated states in the training distribution; Cal-QL calibrates offline value initialization for subsequent fine-tuning [16, 126]. Semi-structured dynamics models and real-world quadrotor learning place structure in predictions of physical transitions [92, 139]. Behavior initialization, value calibration, replay, and dynamics learning solve different parts of the transition to deployment. Reusing prior data can improve sample efficiency, but its usefulness depends on coverage of the states and dynamics encountered during current practice.

Expanding a repertoire through generated experience. RoboCat connects task specialization to generalist retraining [19]. A goal-conditioned generalist is adapted to a new task using 100–1000 teleoperated demonstrations. The resulting specialist generates additional trajectories, which undergo hindsight goal relabeling and, where needed, learned success filtering. These data are combined with the demonstrations and previous training data to train the next generalist. This separation lets a specialist concentrate on a new skill while generalist training revisits both earlier and newly collected tasks. A controlled comparison on four tasks isolates the contribution of self-generated data, and larger experiments incorporate multiple real-robot tasks across embodiments. The reported self-generation iteration depends on human demonstrations for new-task seeding and on task-specific success detection.

SOAR instead targets autonomous collection in configured tabletop scenes [212]. A vision-language model selects a feasible task from a prespecified repertoire using the current scene. A language-conditioned image generator then proposes visual subgoals, which a low-level goal-conditioned policy attempts to reach. After a failure, proposing another task from the resulting scene can sustain collection without restoring the original object configuration. A vision-language success detector selects semantically relevant trajectories, but policy training uses goals relabeled from states actually reached. This decoupling limits the effect of incorrect language or success labels on the action-learning objective. The image generator and task proposer remain frozen while the goal policy learns. The route is well suited to reversible tabletop interactions; its coverage is shaped by the configured repertoire and the feasibility of continuing from the states produced by practice.

Scheduling and distributing practice. AutoRT proposes and schedules tasks for a robot fleet, combining autonomous execution, teleoperation, and safety rules [8]. Embodied foundation-model practice automates reward and success labeling through a frozen earlier checkpoint, examined in Section 4.2 [147]. A criticality-model preprint selects failure-prone states from policy outcomes [63], while USER addresses asynchronous collection and policy synchronization [198]. These components determine different aspects of data quality: whether a task is feasible, whether a visited state is informative, whether its outcome is labeled consistently, and which policy version generated it. Scaling collectors is useful when these interfaces deliver experience that the learner can exploit.

Practice can be supported by another machine or a learned action prior. Robot-Trains-Robot uses a robotic-arm teacher to protect, perturb, reset, and train a humanoid [70]. URSA studies direct real-hardware skill acquisition; SLAC uses a simulation-pretrained latent action space for whole-body real-world RL [57, 69]. A teacher changes which experiments are physically feasible, while a latent action space changes which behaviors are easy to explore. These approaches suggest treating the practice apparatus and action representation as design variables alongside the learning algorithm.

Virtual experience as a proposal source. RoboGen and GenSim generate simulated tasks, while Eureka searches reward programs through simulated RL [175, 172, 114]. RoboCasa, RoboTwin, and RoboTwin 2.0 diversify simulated settings; visual and dynamics randomization target transfer variation [128, 124, 29, 162, 133]. DayDreamer learns predictions from physical interaction and trains through imagination; RISE emphasizes compositional imagined rollouts [179, 191]. Virtual practice increases search breadth, whereas physical practice grounds the consequences of selected changes. A promising allocation strategy uses inexpensive rollouts to explore many repairs and hardware trials to resolve uncertainty about contact, recovery, and transfer.

A failure can nominate new experiences without becoming a new physical trial. Fail2Progress generates simulation settings targeted to observed failures to improve a skill-effect model, whereas MILES collects self-supervised experience around a single human demonstration and an initial reset [74, 131]. The two mechanisms target different scarcity constraints: virtual failure variation versus additional autonomous collection around a prepared real task. A log that labels both only as self-generated data loses the distinction between the source of a trajectory and the source of its training objective.

The value of autonomous collection depends on the learning bottleneck. Across real and simulated tasks, one study found only modest gains from scaling autonomous imitation data in some settings, while additional human demonstrations often helped more; environment design, reset, and success detection remained limiting costs [123]. This suggests a collection strategy that diagnoses whether progress is limited by coverage, data quality, or recovery. More rollouts address the first only if they visit useful states; teaching can address missing behavior; improved preparation and feedback can make otherwise uninformative practice usable.

LiReN uses a higher-level supervisor to propose navigation tasks and calculate rewards during deployed policy learning [157]. RLDG produces stronger task trajectories through reinforcement learning for subsequent generalist-policy distillation, whereas ReSteer generates data around states in which a multitask policy is difficult to steer [184, 33]. ReSteer's principal quantitative experiments are simulated, with a separate hardware test. The three approaches concentrate experience through different selection signals: a supervisor's task proposals, a specialist's acquired competence, or identified steering difficulties.

For physical RSI, these collection mechanisms expose a concrete learning problem: allocate practice among new coverage, correction of weak behavior, and recovery from difficult states. A useful improvement policy must relate this allocation to later gains and to the physical and human effort consumed. The mechanisms above provide actionable choices for such a policy, including demonstration requests, autonomous repetition, hindsight relabeling, and virtual failure variation.

4.2 Self-Evaluation and Verification

Feedback serves two functions in an embodied improvement system. A *search signal* ranks actions, trajectories, or candidate updates during learning; an *acceptance signal* determines whether the resulting version should replace its predecessor. Their information requirements differ. A progress estimate can guide exploration before success becomes observable, whereas deployment acceptance needs a stable task measure and observations of consequential failures such as excessive force, object damage, or delayed breakdown.

Detecting, diagnosing, and anticipating outcomes. Vision-language success detectors assign outcome labels; AHA explains manipulation failures using trajectory evidence and simulated perturbations [44, 45]. FOREWARN ranks predicted action consequences [180]. These methods provide different information at different times. Outcome labels select completed trajectories, diagnoses identify plausible causes of failure, and anticipated consequences screen actions before a costly trial. Their value depends on the response they support: more precise completion detection helps filtering, while a causal diagnosis is more useful for selecting a repair.

VERITAS provides a concrete bridge from action screening to training data [203]. At the start of a task, a vision-language model identifies the target object and proposes a static sequence of pixel-space waypoints. During execution, the policy samples several action chunks; projected end-effector trajectories are scored by their geometric distance from this visual trace, and the best candidate is executed. The model is therefore queried to construct the trace, with inexpensive geometric checks used for subsequent selection. Successful steered robot trajectories are later used for offline behavior-

cloning fine-tuning. Real manipulation experiments evaluate both steering and the resulting policy. Sampling more candidates increases inference cost, while distillation can carry selected behavior into the policy; the visual trace's accuracy and geometric representation determine which mistakes this verifier can detect.

Runtime monitoring distinguishes erratic action streams from stalled task progress. Sentinel separates temporal inconsistency and progress failures, while uncertainty-aware detection targets warning without a labeled failure set [4, 185]. This distinction affects recovery: action instability favors a control fallback, whereas stalled progress may require a new plan or additional task experience. Evaluating a detector thus requires attention to warning timing and the recoverability of the states in which it fires.

Fixed and adaptive progress estimates. MEDAL++ constructs a visual reward from demonstration states, while mobile-manipulation RL combines semantic and low-level rewards [149, 121]. Self-Improving Embodied Foundation Models learn a reusable progress representation during supervised fine-tuning: alongside action prediction, the model predicts the remaining steps to a goal [147]. A separate frozen checkpoint then supplies a reward equal to the decrease in expected steps-to-go between consecutive observations; a small remaining-step estimate also detects completion. The action policy collects trajectories and is updated with on-policy REINFORCE, after which the collection buffer is cleared. Real LanguageTable experiments use this feedback across multiple stations, with a human available for abnormal configurations and resets. Freezing the feedback model keeps its interpretation consistent while the policy changes; the scope of useful learning still depends on what the visual progress representation recognizes.

VIP and LIV also extract goal-relevant reward information from video, reducing the need for manually designed dense signals [112, 113]. Progressor explicitly models progress as a distribution conditioned on initial, current, and goal images [13]. Temporal positions in expert videos provide self-supervised training targets, and predicted uncertainty discounts ambiguous observations. During online RL, an adversarial push-back objective reduces progress estimates on policy-generated observations while expert-video training anchors the model. This addresses the distribution shift from expert behavior to early exploratory rollouts. The online refinement is evaluated on six Meta-World tasks; the four UR5 hardware tasks instead use the pretrained reward for offline reward-weighted action learning from successful and failed demonstrations. The ablation also identifies a tradeoff: persistent push-back can penalize a policy after its behavior has approached the expert distribution.

Visual goal learning offers a different source of supervision. Act2Goal pairs observed actions with hindsight visual goals and uses predicted intermediate states to guide execution; its retained update mechanism is detailed in Section 4.3 [211]. Relabeling treats an achieved outcome as a valid training target even when the commanded task fails. It can therefore make failed practice useful without assigning a scalar reward to every state, provided the policy already produces meaningful interactions.

Language-conditioned feedback offers another form of reuse. ReWiND learns a reward and offline policy from initial data, then adapts to new instructions without a fresh demonstration for every task [202]. A shared representation amortizes feedback construction across related tasks. Its utility depends on whether the new instruction and observed behavior fall within the distinctions represented by the learned reward.

Accepting an update independently. Human comparisons can provide a feedback channel separate from an automated robot reward, albeit at annotation cost [37]. Inverse reward design and reward-gaming analyses show why a training proxy need not coincide with the intended objective, and optimization against a learned reward model can eventually diverge from an independent target under the assumptions studied there [60, 153, 55]. For physical deployment, task completion and safety should also be kept distinct: OopsieVerse measures simulated damage alongside manipulation performance, and TAIL-Safe illustrates runtime monitoring as a separate protective layer [15, 6]. Constrained policy optimization treats expected cost as distinct from reward, but its optimization guarantee should not be mistaken for a certificate of safety under all real-world distribution shifts [1]. A stronger acceptance protocol retains an external task measure and a safety gate across generations, audits cases that the training verifier confidently mislabels, and reports how much human review remains.

The RoboReward preprint emphasizes failed and near-failed outcomes in vision-language reward learning [90]. Such examples retain distinctions that binary success labels collapse together. For

physical RSI, the central opportunity is to learn when each feedback representation is useful: a frozen progress model supports consistent policy optimization, an adaptive estimator addresses newly visited states, and hindsight goals recover supervision from achieved outcomes. Combining these signals with failure diagnoses can guide subsequent learning actions, while an independent acceptance measure tracks whether the changes improve deployment.

4.3 Persistent Self-Modification

Persistent self-modification determines where new experience enters the system. RT-2, OpenVLA, and Diffusion Policy supply different action-model substrates [22, 87, 36]. Learning can update their parameters, a supporting memory, a predictive model, or the surrounding control and training code. The choice matters because these objects carry different information across deployments and require different amounts of data to repair.

Generalist consolidation and local adaptation. The collection routes in Section 4.1 differ in how new behavior enters shared parameters. RoboCat incorporates specialist-generated data into a new generalist; SOAR updates the goal-conditioned control component while keeping its semantic task and subgoal models frozen [19, 212]. The first expands a training mixture across embodiments and tasks, while the second specializes a reusable control policy to newly collected scenes. Self-Improving Embodied Foundation Models update the action model using fixed model-derived feedback; mobile-manipulation RL, SELFI, and HIL-SERL retain policies learned through more local deployment [147, 121, 67, 110]. Their data, compute, and intervention budgets differ substantially. A meaningful comparison follows the scope of the retained change and the deployment conditions under which it is evaluated.

Act2Goal separates a large pretrained model from small, frequently updated adapters [211]. Its goal-conditioned world model predicts intermediate visual states, whose features condition the action model. A multi-scale temporal representation combines dense near-term actions with sparse distant predictions; only near-term actions are executed, preserving local responsiveness while distant states provide goal guidance. During deployment, observed transitions are stored and their outcomes relabeled as visual goals. The associated actions become supervision for a flow-matching update of LoRA parameters, with the base model frozen. This limits update cost while retaining the visual goal interface. Evaluation includes whiteboard writing, dessert plating, and plug-in manipulation on an AgiBot Genie-01, with online real adaptation reported for novel drawing and plug-in tasks. The multi-round quantitative comparison in RoboTwin 2.0 additionally shows that failed rollouts can supply useful hindsight targets. Meaningful initial interaction and occasional real-environment adjustment remain important to this adaptation route.

Corrective supervision. Human language corrections improve a long-horizon robot controller, and TRANSIC learns from human corrections to close a sim-to-real gap [151, 83]. DAgger queries an expert on states induced by the learner, aligning corrective labels with the distribution encountered at deployment [142]. Diffusion Meets DAgger targets distribution-shift errors with generated corrective data in eye-in-hand imitation, while set-supervised diffusion uses differences between a proposed action and a human correction [205, 98]. Human corrective actions, synthetic counterexamples, and robot-collected outcomes contribute different information. Their common technical purpose is to concentrate parameter changes on errors that the original training distribution left unresolved.

State beyond policy weights. RoboEXP retains an interaction-derived scene graph for later manipulation [81]. DayDreamer and RISE make predictive models part of learning, using real experience and compositional imagination in different proportions [179, 191]. Self-modeling and damage adaptation repair software representations or select compensatory behavior after the machine changes [18, 39]; rapid motor adaptation estimates factors needed for locomotion under varying dynamics [89]. Voyager provides a digital example of reusable executable skills [170]. These mechanisms preserve different kinds of information: object relationships in memory, physical transitions in predictive models, and reusable behavior in executable skills. Matching the repair object to the failure can reduce the data and interference associated with changing a shared policy.

Balancing retention and adaptation. Parameter regularization, episodic-memory constraints, and replay preserve earlier capabilities through different mechanisms [88, 107, 141]. LIBERO exposes sequential robot-task transfer; language-guided skill acquisition adds capabilities over time; preserving VLM abilities during action training protects a different representation [105, 34, 61]. The practical

design choice is where to place plasticity. Task-local adapters and memories can absorb frequent changes, while shared policies and representations require evaluation across earlier tasks. Replay spends storage and update computation to revisit those tasks; constraints limit changes that conflict with previously learned behavior. Both must balance retention against the ability to correct a newly encountered weakness.

ConRFT uses offline and online reinforcement fine-tuning of a VLA, with human intervention during real exploration; RLDG distills task-specific RL-generated data into a generalist tested on precise real-world manipulation [30, 184]. The first adapts the deployed model, while the second transfers a specialist's improved behavior. Towards Long-Lived Robots studies continual VLA improvement and retention through reinforcement fine-tuning with reward construction that does not require online environment feedback [106]. Together with LIBERO [105], these methods motivate separating rapid local learning from consolidation into shared capabilities.

The corresponding physical RSI decision concerns the scope and timing of a repair: update a task-local adapter, revise a model or memory, or consolidate new behavior into shared parameters. Learning this decision requires records of what changed, which weakness motivated the change, and how both new and retained capabilities performed afterward. The methods above supply distinct update operators and make their different data and retention costs visible.

4.4 Closing the Physical Improvement Loop

A physical improvement loop links an earlier deployment to a retained successor and another opportunity to practice. Table 3 compares how representative systems connect collection, feedback, updating, and redeployment. It highlights the interface each route makes reliable and the supervision or recovery on which that interface depends.

The table reveals three routes to closing the loop: expand a reusable policy with new data, adapt a local learner through continued practice, or revise experiment and training code. RoboCat's specialist-to-generalist route broadens the repertoire while retaining earlier data [19]. SOAR emphasizes local scene coverage: scene-specific autonomous-data co-training raises mean success from 28% to 57% across ten evaluation scenes [212]. Its released approximately 30.5K trajectories also include additional collection and constitute a separate resource from that comparison. These results address different learning goals, so policy scope and the data mixture help interpret their gains.

The routes also move costs between stages. MEDAL++ invests in learned recovery, while HIL-SERL uses human corrections to redirect exploration [149, 110]. Frozen-feedback practice reuses a stable progress model, while Act2Goal extracts new goal supervision from observed transitions [147, 211]. VERITAS spends inference computation on candidate screening and then distills the resulting experience; RISE spends more of its improvement budget on imagined trajectories supported by physical data [203, 191]. ENPIRE allocates real trials among editable training recipes [181]. USER and autonomous-collection analyses show why synchronization, preparation, and recovery affect the usable practice budget [198, 123].

For physical RSI, closing these interfaces creates a basis for choosing the next improvement action from the observed bottleneck. Broad retraining needs decisions about mixtures and retention; online adaptation needs sustained recovery and informative feedback; code revision needs credit assignment among experimental changes. Execution traces can support these choices by recording recurring states, failed actions, and successful recoveries. Table 1 supplies a common vocabulary for the retained deployment and process changes.

4.5 Process-Level Candidates for Physical RSI

Three editable decisions emerge from these methods: *what to practice*, *how to judge it*, and *how to change the learner*. RoboCat's growing generalist supports more efficient task adaptation [19]; SOAR, frozen-checkpoint practice, and Act2Goal connect task proposals, feedback, and hindsight goals to later policy updates [212, 147, 211]. Our view is that the strongest opportunity lies in coupling these decisions. The value of a proposed task depends on the available feedback and update operator; an apparently informative failure may be wasted if the system changes the wrong component.

Candidate edit spaces differ in their functional role. RoboGen and Eureka generate simulated tasks or rewards, while adaptive goal generation, regret-based design, and prioritized replay select

Table 3: Representative physical improvement routes. Each row identifies the experience used for learning, the signal applied to it, the component that retains the change, and the reported hardware setting and operational dependencies. Results across different tasks and budgets are not directly interchangeable.

System	Improvement experience	Training or selection signal	Retained change	Hardware setting and dependencies
MEDAL++ [149]	Alternating real task and undo trials	Demonstration-derived visual reward	Task and undo policies	Four Panda tasks; initial demonstrations, setup, occasional resets
HIL-SERL [110]	Real trials and human corrections	Sparse success reward; mixed replay	Task policy	Precision, dynamic, bimanual manipulation; classifier data, corrections, resets
RoboCat [19]	Specialist-generated real and simulated trajectories	Hindsight goals; learned success filtering	Retrained generalist	Multiple embodiments; new-task demonstrations, one self-generation iteration
SOAR [212]	Scene-conditioned autonomous trials	VLM filtering; hindsight goal supervision	Goal-conditioned policy	Tabletop scenes; configured task repertoire, continuation from current scene
Embodied FM practice [147]	On-policy robot trials	Frozen steps-to-go reward and completion estimate	Action policy	LanguageTable stations; human monitoring and abnormal-state recovery
SELFI [67]	Real visual-navigation experience	Online RL after model-based initialization	Navigation policy	Navigation around people; safety intervention
Act2Goal [211]	Self-collected transitions	Hindsight visual goals; flow matching	LoRA parameters	Genie-01 adaptation; initial competence, occasional environment adjustment
VERITAS [203]	Successful steered robot trajectories	Geometric action screening; later behavior cloning	Fine-tuned policy	Real manipulation; visual traces, candidate inference, successful-rollout selection
RISE [191]	Primarily imagined trajectories	Compositional world-model estimates	Updated policy	Bimanual real-task tests; physical warm-up and corrections
ENPIRE [181]	Real trials of agent-written recipes	Fixed success API; experimental outcomes	Policy, code, recipe summaries	Preprint; YAM fleet, human-assisted API setup and reset design

experience through distinct signals [175, 114, 50, 41, 82]. Digital algorithm and workflow search expose optimizers or scaffolds to revision [137, 71, 201]. The ASPIRE and SHAPER preprints emphasize control-program, skill or harness changes, with principally simulated or benchmark evidence [108, 173]. An execution scaffold can become retained state in X; an improvement scaffold determines how later updates are generated or selected. A program's editability does not settle which role was tested, so comparisons should identify its inputs, persistent decisions, and downstream learning consequences.

ENPIRE provides a concrete architecture for editable learning decisions [181]. In its first stage, coding agents and human feedback construct task-specific safety, reset, and verification modules, which then become fixed Gym APIs. Its second stage grants agents write access to policy and training code. Rollout videos, proprioception, and success outcomes guide revisions to behavior-cloning or RL recipes, including replay choices and update hyperparameters. Parallel agent–robot pairs test hypotheses asynchronously and share successful code changes. Push-T and pin-insertion experiments examine policy discovery and fleet scaling; larger fleets reduce time to a target outcome while increasing token cost and reducing per-robot utilization. This architecture makes physical experiment selection, code revision, and resource allocation explicit parts of the learning process.

The preprint's current version also reports reuse of accumulated research experience across tasks [181]. After pin-insertion experiments, agents summarize the evolution of their recipes; that written summary is supplied when beginning GPU-insertion research. Prior raw trajectories, transient logs, and checkpoints are removed, locating the transferred information in an explicit account of the earlier experiments. The reported GPU-insertion outcome therefore illustrates a retained channel for improvement knowledge alongside policy and code updates. It motivates measuring which parts of that knowledge improve subsequent recipe selection, how well they transfer to different failures,

and what trial and computation budgets they save. Section 5.3 develops the controlled comparisons needed to examine those effects.

5 Resources and Evaluation for Physical Improvement

Studying physical improvement requires resources for acquiring experience, learning from it, and testing the retained result. These capabilities are distributed across datasets, collection software, simulators, and evaluation systems. We first compare what these resources make possible, then examine the questions current evaluations answer. Their coverage gaps motivate the prospective process comparison proposed in the final part of this section.

5.1 Resource Capabilities and Experimental Scope

Physical data and collection interfaces. Reusable robot corpora provide behavioral coverage; teaching interfaces determine how new demonstrations enter that coverage; collection systems connect task execution to training and deployment. Table 4 compares these functions. A corpus supports controlled reuse of existing observations and actions, whereas an interactive collection system also permits changes to the tasks, policy, and operating conditions that generate the next sample. The distinction determines which learning decisions a research platform can expose.

Table 4: Physical data and collection resources grouped by experimental capability. Entries compare data access, feedback interfaces, and retained artifacts within each resource's reported experimental scope.

Capability	Representative resources	Interaction and feedback	Retained artifact and physical scope
Reuse robot experience	RoboNet, BridgeData V2, DROID; AgiBot World preprint [40, 167, 86, 24]	Recorded manipulation under different robot and collection scopes; AgiBot includes human verification	Training data for later policies; robot, task, and collection provenance determine transfer scope
Acquire demonstrations	Robomimic, ALOHA, UMI, RoboMIND, FastUMI [117, 209, 35, 178, 210]	Offline human data, robot teleoperation, or portable demonstration interfaces	Demonstrations and imitation policies; human control and robot execution remain distinct acquisition modes
Reuse human activity	EgoVerse; RoboWheel [136, 206]	Egocentric human data; hand–object video reconstruction and simulated retargeting	Human-source or generated supervision; RoboWheel separately reports real-robot tests
Coordinate collection	AutoRT; USER [8, 198]	Task proposals and fleet execution; asynchronous collection, buffering, and policy synchronization	Trajectories and deployed checkpoints; teleoperation, safety rules, and recovery affect collection throughput
Reuse software interfaces	SERL, LeRobot, RIO [109, 25, 130]	RL software, learning tooling, and robot I/O interfaces	Reusable training and deployment components; task setup and feedback still depend on the experiment
Study whole-body tasks	BEHAVIOR Robot Suite [84]	Household whole-body manipulation on a physical platform	A physical task and system setting for studying execution and learning requirements
Connect policy generations	RoboCat; SOAR [19, 212]	Earlier-agent data for generalist training; model-guided collection for instruction-policy learning	Later policy parameters; mixed real/simulated data in RoboCat and real scene-local practice in SOAR

Broadly trained action policies supply a starting point for experiments with limited local data [21, 160, 87]. Subsequent collection must supply information missing from that starting policy. Demonstration-quality estimation ranks individual examples, domain reweighting changes data mixtures, influence-based selection targets downstream imitation outcomes, and retrieval supplements a small new-task set from prior data [66, 65, 5, 182]. These methods act on different units: an example, a domain, or task-specific support. A useful comparison holds the candidate data available to the selector constant and measures the retained policy obtained from its chosen subset.

Coverage also depends on the variation represented within a dataset. Factor-compositional collection and cross-embodiment training examine how task factors and robot domains affect transfer; auxiliary-modality co-training asks which additional signals help a target policy [54, 192, 102]. Action-unlabeled video offers another source of coverage, while shortcut-learning analysis shows why fragmented component datasets can preserve reliance on incidental features [155, 183]. Increasing sample count, combining domains, and adding modalities therefore address different bottlenecks. Their value depends on the deployed task's missing information and on the compatibility of the collected observations and actions.

Sustained physical collection adds a recovery constraint. Task-and-undo learning creates further practice opportunities by restoring useful states; mobile-manipulation and navigation learning combine continued interaction with different behavior priors, rewards, and supervision [149, 121, 67, 157]. Real grasping, walking, and manipulation RL demonstrate learning under different physical protocols, while intervention-based RL trades human corrections for more directed exploration [85, 59, 91, 110]. The resulting throughput depends on how often an attempt ends in a learnable, recoverable state. Analyses of autonomous data scaling accordingly identify environment preparation, reset, and success detection as substantial experimental inputs [123]. A platform that collects rapidly but repeatedly needs manual restoration may supply less usable learning experience per hour than one with slower, sustained practice.

Virtual and generated experience. Virtual resources expand the range of candidate tasks and updates that can be screened before hardware use. Table 5 separates interactive simulation, generated demonstrations, rendering, and model imagination because their feedback supports different inferences. A simulator can return state transitions or task outcomes under its dynamics; a rendered trajectory mainly supplies observation variation; an imagined rollout depends on a learned predictor. These capabilities should guide the choice of a screening experiment.

Table 5: Virtual and generated resources for expanding training and screening. The physical scope identifies where a paired hardware test or additional fidelity check is needed.

Capability	Representative resources	Feedback and supported update	Physical scope
Vary scenes and tasks	RoboCasa, RoboTwin, RoboTwin 2.0 preprint [128, 124, 29]	Simulated household or dual-arm interaction for training and evaluation	Task and scene variation under simulated dynamics; hardware transfer requires separate testing
Generate learning problems	RoboGen, GenSim, Eureka [175, 172, 114]	Generated tasks or reward programs evaluated through simulated learning	Searches task and supervision choices; physical feasibility and candidate ranking remain transfer questions
Expand seed demonstrations	MimicGen, DemoGen, LodeStar, constraint-preserving generation, R2RGen [118, 188, 168, 103, 187]	Synthetic or transformed supervision for policy training	Additional examples derived from seed data; physical testing measures the resulting policy
Render or reuse digital twins	Real2Render2Real; CASHER; GS-Playground [194, 164, 80]	Rendered trajectories, twin-based simulated data, or high-throughput photorealistic observations	Visual coverage and dynamics fidelity are separate properties; rendering throughput alone leaves contact behavior unresolved
Learn through imagination	DayDreamer; RISE [179, 191]	Predictive models support policy training through imagined rollouts	Physical data and endpoint tests ground the model's use; imagination has its own sample and compute cost
Simulate interactive worlds	Interactive world simulation [174]	Action-conditioned generated rollouts for training and evaluation	Action and contact fidelity must be tested for the intended hardware use

Visual and dynamics randomization address different sources of transfer error, and humanoid dexterous manipulation provides a setting in which a virtually trained policy is tested on hardware [162, 133, 104]. Simulation-based evaluation of real-world manipulation policies examines whether a virtual test reproduces useful policy comparisons [97]. Paired virtual and hardware outcomes can therefore establish where inexpensive screening is informative: object appearance may transfer while

contact, recovery, or failure rankings do not. Physical trials, demonstrations, rendered data, simulated rollouts, and imagined rollouts retain separate counts throughout the comparison.

The retained artifact also varies across resources. Interaction can enrich a scene graph, hindsight goal construction can support policy fine-tuning, and specialist RL data can be distilled into a generalist [81, 211, 184]. Targeted trajectory generation can improve steerability, with its simulated rollout evidence and hardware evaluation serving different roles [33]. Agentic code, skill-library, and harness revision further enlarge the artifact space in recent preprints [181, 108, 173]. Cross-task reuse of research experience adds another resource: the agentic real-robot study reports transfer of Markdown research summaries [181]. Resource comparisons should consequently identify whether an experiment retains data, policy parameters, scene knowledge, a predictive model, research notes, or editable training software. The longitudinal record proposed below connects these artifacts to the experience and decisions that produced them.

5.2 Current Evaluation Targets and Coverage Gaps

Execution, transfer, and retention. Skill selection, generated control programs, feedback-conditioned replanning, and spatial waypoint synthesis provide different execution interfaces [7, 100, 72, 73]. Their evaluation asks whether the available system completes a task, how often it replans or rejects a call, and what latency, compute, intervention, and safety costs accompany execution. A retained update adds a second question: whether the successor improves over its parent and preserves earlier capabilities.

Existing benchmarks supply useful task distributions for these questions. Multi-task and meta-RL manipulation, diverse manipulation tasks, language-conditioned sequences, everyday activities, and long-horizon reasoning emphasize different generalization axes [195, 77, 119, 95, 58, 204]. Sequential-task transfer makes forgetting observable; continual VLA fine-tuning, language-guided skill acquisition, and preservation of VLM abilities during action training examine different forms of retention [105, 106, 34, 61]. Parameter regularization, episodic-memory constraints, and replay are retention mechanisms whose effects must be checked on the capabilities intended to survive [88, 107, 141]. A held-out task score and a post-update retention score thus answer complementary questions about the policy.

Focused tests expose failures hidden by an aggregate completion rate. Whole-body locomotion and manipulation, visual and scene perturbations, procedural physical reasoning, low-level manipulation reasoning, and task-generalist simulation each require different observations and controls [148, 135, 75, 208, 190]. Adversarial testing of legged policies targets long-tailed vulnerabilities [150]. These axes help diagnose whether a weak result comes from perception, reasoning, control, or physical robustness; collapsing them into one score obscures the repair a learning system should attempt.

Outcome measurement and uncertainty. Training signals vary in what they observe. Visual success classification labels an outcome; failure diagnosis explains a trajectory; prospective screening and action verification rank proposed behavior; learned reward models assign an optimization signal [44, 45, 180, 203, 90]. Video-derived reward or value representations extend supervision across visual or language goals [112, 113]. Online progress refinement is evaluated in Meta-World simulation, while the same reward-estimation study tests offline RL on a physical UR5 [13]. Human comparisons offer another signal at annotation cost. Inverse reward design, reward-gaming analysis, and proxy-reward experiments show under their respective assumptions why optimization feedback can diverge from the intended outcome [37, 60, 153, 55]. An independent outcome measurement must therefore accompany policy evaluation, especially when training also refines its verifier. Raw outcomes and independently labeled failures permit checks of confident errors and newly induced behavior.

External comparison can use distributed double-blind robot tests, automated real-world evaluation, real-to-sim assessment, or statistical prediction of sim-to-real performance [12, 213, 76, 31]. These routes trade hardware access against the assumptions needed to predict its outcomes. Small-sample stopping rules allocate expensive trials between candidates; contrast sets reveal failures outside narrow language-conditioned tests; offline discounted-liveness estimates use existing logs when fresh rollouts are scarce [154, 10, 171]. Virtual and offline estimates need calibration against held-out hardware outcomes, while physical comparisons need uncertainty estimates and explicit stopping rules. Neither a large virtual test set nor repeated use of the same real test removes selection effects.

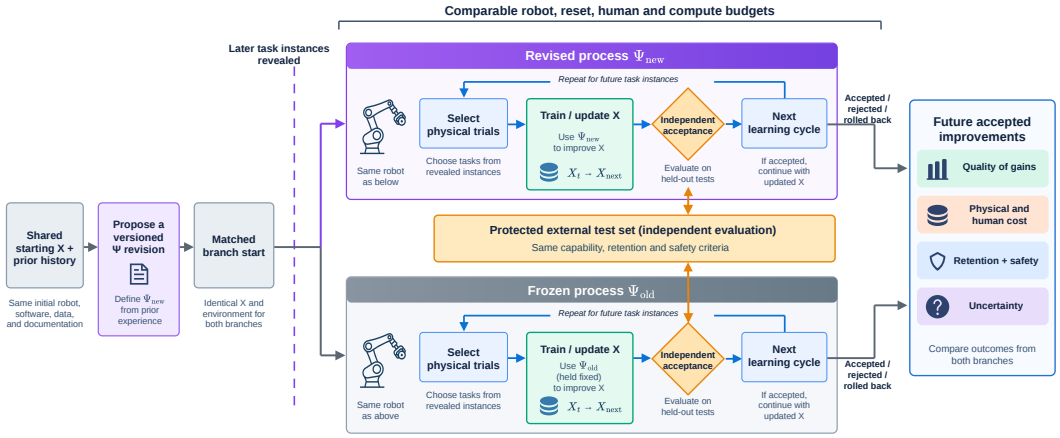


Figure 4: Proposed future-learning comparison of revised and predecessor processes. Both branches update X from matched starting states under common resources. Deployment gates govern intermediate updates; a sealed endpoint audit measures retained capability, retention, and safety. Resource and failure records accompany the comparison.

Task completion also leaves damage and safe recovery partly unmeasured. Simulated damage accounting separates mechanical, thermal, or fluid harm from completion; runtime monitoring adds a protective interface for deployed imitation policies [15, 6]. Safe-RL test suites, constrained optimization, generalized safe exploration, uncertainty-aware safe learning, and recovery policies address different risk quantities and assumptions [79, 1, 166, 11, 161]. Comparisons should keep harm severity, human stops, recovery, rollback, and measurable wear alongside task capability. A higher completion rate obtained through more unsafe exploration has a different resource and deployment meaning.

The missing process comparison. Task behavior, retained-policy quality, and risk measurements leave open the value of a saved improvement rule for *later learning*. Current task success can rise through additional data, better resets, stronger initialization, or a changed task mixture. Establishing the value of a process revision requires controlling these inputs while allowing both branches to learn. This motivates the protocol below: the endpoint is retained capability after common future learning opportunities, with retention, harm, and resource use reported alongside it.

5.3 A Prospective Comparison of Improvement Processes

We propose a branching comparison of a revised improvement rule and its saved predecessor. Policy gains obtained from earlier-agent data, autonomous collection, frozen-model feedback, intervention-based fine-tuning, and specialist distillation motivate distinct system-level baselines [19, 212, 147, 30, 184]. Agentic training-code revision motivates testing the learning procedure itself [181]. The resulting process effect is measured on future opportunities revealed after the revision, rather than on the task or score that induced it. Figure 4 summarizes the matched starting state, learning branches, and endpoint measurement.

Save the rule and match the starting conditions. Register the rule's inputs, outputs, persistent parameters or code, permitted edits, and the history inducing its revision. Save Ψ_t before revision and branch from matched X and history: one branch uses Ψ_{t+1}, while the other retains Ψ_t. Replay the rules on the same X, history, candidate pool, and seeds; for stochastic rules, compare output distributions to establish which registered decision changed. Freeze only the predecessor's improvement rule; both branches may update their action policy. Adaptive goal selection, regret-based design, and prioritized replay illustrate why a fixed rule can still respond to changing inputs [50, 41, 82]. Later task instances are then revealed under common resource ceilings, external criteria, and stopping rules. Held-out procedural testing supplies a generalization analogy, while statistical self-modification guarantees remain conditional on their assumptions [38, 169].

Keep optimization, deployment, and endpoint measurement identifiable. Training feedback guides learning; a separately governed deployment gate accepts or vetoes candidates and can return feedback

for subsequent learning; a sealed audit measures the endpoint after systems and analysis are locked. Repeated gate access can influence the learner, so preserve untouched audit blocks and bridge tests when measurement criteria change. The gate's versions, returned information, query access, and human contributions belong in the longitudinal record. This arrangement permits adaptive practice and intermediate deployment while retaining a common endpoint measurement.

Evaluate at a common horizon, including rejected updates. Preregister a budget vector $B = (B_{\mathrm{robot}}, B_{\mathrm{compute}}, B_{\mathrm{human}})$ and K common future learning opportunities. Keep physical-trial counts, elapsed time, resets, and harm limits explicit. Let $X^b_{B,K}$ be the operating state retained by branch b at this horizon: rejection leaves the parent deployed, rollback restores the last authorized state, and an early budget stop carries that state forward without extra trials. Prespecify a capability score $C_{\mathcal{T}}$ and task weights on the sealed audit suite $\mathcal{T}$. For $G^b_{B,K} = C_{\mathcal{T}}(X^b_{B,K}) - C_{\mathcal{T}}(X_{\mathrm{start}})$, define the paired process effect as

$$\Delta_{\Psi}(B, K) = G^{\mathrm{new}}_{B,K} - G^{\mathrm{old}}_{B,K}. \tag{3}$$

The endpoint includes zero or negative gains and branches with no accepted update. Evaluate saved baseline and endpoint systems in matched external blocks, and replicate paired comparisons to estimate uncertainty. On shared hardware, randomize order within blocks and account for calibration, wear, reset, and residual scene state. Retention and harm remain separate outcomes and constraints. Attempt and acceptance rates, learning curves, and cost to a capability target describe how a branch reaches its endpoint; their denominators and censoring must be reported.

Process-development resources B_{search} are accounted for separately from subsequent-use resources B, including failed proposals and human assistance. Equation (3) measures the future benefit of a saved rule; lifecycle efficiency also depends on how many uses amortize its development. Allocate audit costs consistently between branches. Report physical practice, simulation, imagination, compute, demonstrations, reset labor, and intervention as distinct quantities so that an expensive revision and a reusable efficient rule can be compared on the same terms.

> **Illustrative grasping comparison.** Allow revision of the diagnosis-to-practice rule while holding the optimizer, training feedback, reset protocol, and deployment gate fixed. Save both rules and start from matched X and history before revealing new cup instances. Both branches learn under common B, K. On shared hardware, randomize trial order within matched blocks and record calibration, wear, and residual state. Lock endpoint systems, conduct the sealed audit, and compute Equation (3) alongside retention, harm, and search/use costs. Repeating the comparison across process versions tests whether intervention knowledge accumulates.

5.3.1 Improvement Trajectories as the Benchmark Unit

The proposed benchmark follows an improvement trajectory from versioned X and Ψ through experience selection, candidate changes, acceptance or rejection, and subsequent deployment. Table 6 consolidates the records needed for the protocol. A version graph keeps rejected candidates attached to their parent with the consumed budget and rejection reason; accepted candidates advance the operating lineage. Proposed Ψ revisions form separate branches linked to their prospective comparison. This preserves the unsuccessful opportunities behind the endpoint as well as the accepted gains.

A practical first study can expose one decision, such as targeted practice selection, on one robot platform. Virtual tasks, candidate rewards, randomization, and model imagination provide alternative screens; sequential-task tests and damage accounting add retention and harm measurements [128, 124, 175, 114, 162, 133, 191, 105, 15]. Recording the selected candidate before revealing later hardware instances makes failed transfers informative about the screen. The resulting trajectory links resource choice, retained system change, and future process benefit in a single reproducible experiment.

Table 6: Proposed longitudinal records connecting physical trials, retained updates, and improvement-process comparisons. These fields consolidate collection provenance, measurement access, and resource accounting.

Record	Minimum retained information	Comparison supported
Physical trial	Robot, task, pre/terminal state, collecting policy, reset operator, intervention, outcome, safety event, calibration, order, and wear	Capability and harm at matched conditions; real, simulated, rendered, imagined, and human-source experience counted separately
Data selection	Candidate pool, selected/rejected examples, score, selector version, modality, task-factor and embodiment coverage, training exclusions	Downstream value of example selection, mixtures, and retrieval under common available data
Retained update	Parent/candidate X, policy/memory/model/code changes, training provenance, accepted/rejected/rolled-back status, and deployment time	Successor versus parent; continued deployment and retention, including unsuccessful candidates
Measurement	Feedback, verifier and gate versions; queries, returned information, access permissions, independent labels, stopping rule, and sealed audit blocks	Stable endpoint measurement despite adaptive feedback and repeated gate use
Process comparison	Registered old/new rules and edit permissions, inducing history, matched starting X, later tasks, B, K, search/use costs, human effort, and early stops	Retained capability against frozen-Ψ at common horizon, with uncertainty, retention, and harm

6 Opportunities and Challenges of Physical RSI

Sections 4 and 5 compare learning mechanisms, resources, and measurements. We now examine the settings in which adapting the process could be valuable and the interacting physical, algorithmic, and safety constraints that shape those opportunities.

6.1 Opportunities

Sustained adaptation after deployment. Diverse physical data and pretrained action policies provide reusable starting capabilities [86, 21, 22, 160, 87]. Deployment then reveals local weaknesses caused by new objects, workspace changes, wear, calibration, and task instructions. Continued manipulation and navigation learning demonstrate how these outcomes can support retained improvements, with different recovery and supervision arrangements [149, 121, 67, 157, 91]. The opportunity for process adaptation is greatest when the limiting factor changes over time: early learning may need coverage, later learning may need more accurate feedback or a different repair. A process that identifies this shift could allocate practice more effectively than a fixed recipe. Its benefit should include later-task generalization and retention, alongside labeling, reset, supervision, and recovery effort. This makes sustained deployment a setting for studying both changing robot capability and changing learning requirements.

Selective transfer across robots and sites. Broad retraining, scene-specific co-training, and checkpoint-guided practice reuse earlier experience at different scopes [19, 212, 147]. Multi-robot datasets supply coverage, while task orchestration and asynchronous infrastructure connect collection to changing deployed policies [40, 167, 86, 24, 8, 198]. Their collection arrangements include teleoperation and human verification, which affect both data value and cost. Sharing can amortize an expensive failure when another robot encounters a related weakness. The receiving learner needs the collecting policy, embodiment, calibration, task, and outcome conditions to judge relevance; indiscriminate pooling can instead emphasize easy sites or mix incompatible action interfaces. A further research opportunity is to transfer the *decision* learned from a failure: which observations identified its cause and which repair was effective. That relationship may be reusable even when the raw action trajectory is poorly matched to the receiving robot.

Reducing the cost of informative experiments. Experience order is a design variable in curriculum learning [17]. Reachable challenge, regret, and learning potential supply different estimates of task usefulness [50, 41, 82]. Virtual task and reward generation increase the number of proposals, while hindsight supervision and targeted policy refinement increase the uses of collected outcomes

[175, 172, 114, 211, 184, 33]. Criticality modeling adds a failure-prone-state proposal route, with preprint evidence across benchmarks and one real task [63]. The physical opportunity is to allocate hardware to questions whose answers would change an update decision. Virtual trials could screen broad alternatives; real contact or recovery trials could resolve the remaining uncertainty. The useful unit of cost is the complete experiment, including preparation, labeling, reset, human intervention, and discarded trials. A learned selector should connect those costs to later retained gain, so that a cheap but uninformative trial does not displace a more valuable experiment.

Coordinating repairs across components. A failed manipulation may arise from action generation, scene representation, dynamics prediction, or outcome assessment. Interaction-derived scene graphs and predictive control models demonstrate repair targets beyond policy parameters [81, 179, 62, 191]. Failure explanations, predicted consequences, and body or behavior adaptation provide complementary information about where a change may help [45, 180, 18, 39]. The opportunity is to identify the smallest effective intervention and coordinate dependent updates. A representation repair may change policy inputs; a dynamics correction may change imagined training data; a verifier repair may change which trajectories are retained. A bounded study could include calibration or controller-configuration changes and learn when they are preferable to weight updates. Software body-model revision changes the representation of the plant; physical reconfiguration additionally requires renewed identification and capability-and-safety acceptance. Joint adaptation of software and hardware remains a scoped research direction.

6.2 Technical and Physical Challenges

These bottlenecks interact. Automating collection can shift the limiting cost to reset and outcome verification; faster virtual search can worsen candidate selection if its ranking is biased; concurrent fleets can obscure which version produced a gain. We group the following constraints by their physical or algorithmic origin, then examine protection across exploration, deployment, and process editing. A useful repair must improve the whole loop under its resource and risk envelope. Figure 5 illustrates three possible couplings and their intervention points.

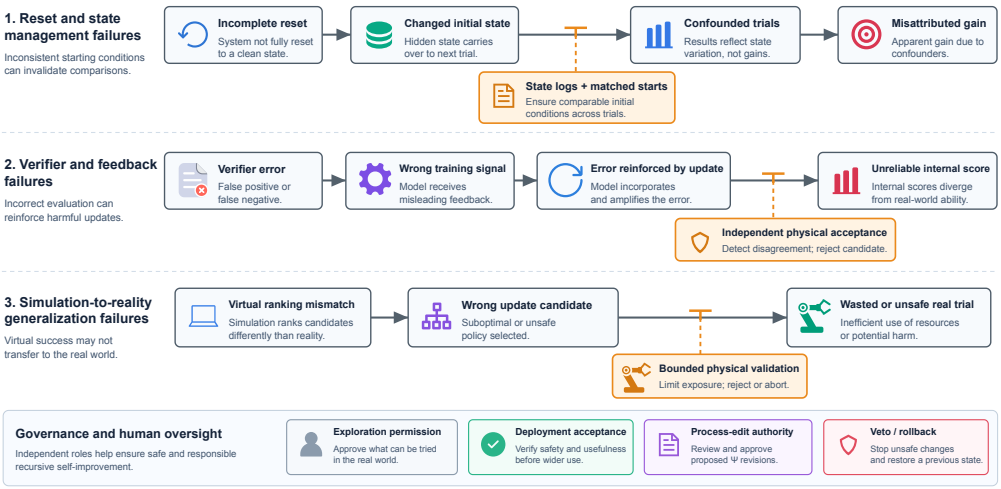


Figure 5: Failure propagation and intervention points in physical improvement. Incomplete resets alter later trials, verifier errors corrupt learning feedback, and virtual ranking errors favor unsuitable updates. State logging, independent deployment checks, and targeted physical validation address these dependencies at different interfaces.

Physical-world constraints on experimentation. *Physical rollback is not checkpoint rollback.* Restoring an earlier policy cannot restore a broken object, undo wear, or recreate a spilled scene. Even without an accident, use, recalibration and environmental change can alter the dynamics faced by the next update. Rapid motor adaptation and physical self-modeling expose the need to respond to changing embodiment and dynamics [89, 18]. Material changes persist through software rollback. A longitudinal comparison must therefore track physical condition, calibration versions, trial order and

recovery actions alongside X and Ψ. On shared hardware, the first branch can change the conditions encountered by the second; on separate robots, plant differences can confound attribution. Matching initial software states is insufficient unless physical starting conditions and residual uncertainty are also characterized. This nonstationarity concerns both the learner and any simulator or safety model used to evaluate it.

Throughput is constrained by the least automated step. Sample-efficient optimization still requires fresh, usable trials. Autonomous-collection analyses identify reset design, scene preparation, and success detection as continuing costs; learned undo behavior improves practice availability where recovery is feasible [123, 149]. Mixed collection, asynchronous infrastructure, and human corrective exploration address task coverage, policy delivery, and difficult states, respectively [8, 198, 30]. Reusable RL software lowers implementation effort, while intervention-assisted learning deliberately spends human attention to improve manipulation [109, 110]. These gains concern different resource channels. Accelerating optimization has little effect on end-to-end throughput when reset or verification dominates; increasing autonomous attempts can increase labor if more episodes need exceptional recovery. This coupling motivates choosing changes from their effect on complete learning cycles, including the physical states left for subsequent practice.

Cheap candidates can have the wrong physical ordering. Simulated task diversity in RoboCasa, RoboTwin, ManiSkill2 and RoboTwin 2.0, automatic task generation in RoboGen and GenSim, and reward search in Eureka enlarge the space of candidate policies or training procedures [128, 124, 58, 29, 175, 172, 114]. Visual and dynamics randomization target particular transfer gaps, while DayDreamer and RISE use learned or compositional models to obtain imagined experience [162, 133, 179, 191]. Humanoid dexterous manipulation trained with sim-to-real RL provides hardware evidence for a transferred final policy [104]. SIMPLER explicitly investigates simulated evaluation of real-robot manipulation policies, underscoring that a virtual screen's predictive value must be measured against hardware outcomes [97]. For selecting an update, a virtual test must preserve the *ranking* of candidate updates well enough that the system spends scarce hardware trials on the right ones. Errors in contact, latency or object state can favor a learning rule that exploits the simulator rather than one that produces better physical policies. The useful screening regime is one in which saved hardware effort exceeds the cost of misleading selections.

A paired screening study must preserve candidate identity, trial conditions, uncertainty, and the physical cost of mistaken nominations. Safety and retention conditions should be checked separately from any capability ranking. The practical endpoint is whether the screen selects useful candidates within the hardware budget, rather than perfect agreement between two complete rankings.

Algorithmic and distributed-learning constraints. *Feedback becomes a moving target as the system changes.* Visual success detectors, failure diagnosis, prospective screening and action verification provide distinct signals; a policy update changes the ambiguous cases they encounter [44, 45, 180, 203]. Video-based rewards from VIP and LIV, Progressor's simulation-tested online refinement, and RoboReward's learned robotics reward model make practice signals denser or more adaptive in different ways [112, 113, 13, 90]. A model trained to label past trajectories need not remain calibrated on behavior induced by later optimization. If its labels also enter the next training set, a misclassification can shape both the policy and the examples used to retrain the verifier. The practical coupling is between exploration and measurement: more self-labeled data may amplify, rather than expose, an error. Independent audits of novel failure modes are needed even when internal feedback remains useful for learning.

Asynchronous fleets can learn from the wrong generation. USER makes concurrent real-robot collection and policy synchronization possible; AutoRT orchestrates multiple physical agents, and DROID demonstrates the breadth of data that different sites can contribute [198, 8, 86]. Those strengths also create version skew: a trajectory may be gathered by an old policy, judged by a newer verifier and used to train a still later policy on a different robot. Such off-policy data are not inherently unusable, but their relevance depends on how behavior, sensing, and task distribution have changed. If a revised Ψ selects data from earlier generations, it may appear efficient because it inherited an easier task mix or a more favorable labeler. Data selection and updating must remain coherent with the policy and evaluation versions that produced the trajectories.

Accumulation can erase what was already learned. LIBERO exposes sequential transfer and forgetting in robot learning, while parameter regularization, episodic-memory constraints, and replay address

different routes to retention [105, 88, 107, 141]. Continual VLA fine-tuning and language-guided skill acquisition concern retention across robot skills; preserving a vision–language model's capabilities during action fine-tuning is a related but different forgetting problem [106, 34, 61]. Physical systems add coupling between policy, world model, scene memory, and embodiment: a new policy may collect data that changes the model, and the model's error may in turn bias the next policy update. RoboEXP's retained scene representation, RISE's imagined updates and damage-adaptation work make these different sources of persistent state concrete [81, 191, 39]. A process that rapidly improves one task while degrading earlier abilities, or that depends on increasingly expensive repair, is not necessarily a better improvement process. The technical problem is deciding when an update should be accepted, rejected or scoped to a subset of tasks without assuming a single latest-task reward captures lifetime capability.

Process edits create a harder attribution problem than policy edits. ENPIRE's preprint lets agents use real-robot feedback to revise training code after people establish a task-specific reset, safety and verification interface [181]. Digital algorithm search and automated agent-workflow design show that optimizers and procedural scaffolds can be legitimate search targets, but their software benchmarks do not settle the cost or reliability of physical process search [137, 71]. A changed code path may alter data collection, training, evaluation and deployment simultaneously; a successful descendant policy then gives little information about which edit was beneficial. Restricting each candidate to a named, reversible process component makes interpretation and rollback more feasible, at the cost of narrowing the search space. Larger edit spaces require correspondingly richer evidence for assigning credit.

6.3 Safety and Trustworthiness

Safety has three boundaries in a changing system: which trials may be attempted, which successor may be deployed, and which parts of the procedure may be edited at all. A guarantee or empirical result at one boundary should not be transferred to another without checking its assumptions.

Complementary learning and safety signals. Inverse reward design treats specified reward as imperfect evidence of intended behavior, reward-gaming analyses characterize ways a proxy can be optimized without satisfying its target, and reward-model overoptimization experiments exhibit divergence under their studied assumptions [60, 153, 55]. Human preference feedback offers another signal but has annotation and consistency costs [37]. In robotics, visual success detection, VERITAS-style action verification and learned reward models can usefully reduce the need for constant human labeling, while an earlier frozen detector can stabilize one part of an autonomous practice pipeline [44, 203, 90, 147]. Deployment acceptance additionally needs physical outcome and safety measurements independent of the optimized score. If Ψ can revise its verifier, the conflict becomes sharper: the candidate may change both the behavior being judged and the instrument that judges it. Independent physical outcomes and a separate acceptance authority keep this measurement stable across changes to the learner.

Safe exploration and safe redeployment are different problems. Constrained policy optimization controls an expected cost in its optimization setting; generalized safe exploration and ActSafe provide further constraint-aware learning mechanisms under their respective assumptions, while Recovery RL learns a distinct recovery channel [1, 166, 11, 161]. Safety Gymnasium supplies simulated comparisons of safe-RL methods [79]. TAIL-Safe studies task-agnostic runtime monitoring for imitation policies, whereas OopsieVerse measures damage in simulated manipulation [6, 15]. EigenSafe studies probabilistic safety assessment under stochastic dynamics [78]. During learning, a robot needs limits on attempted trials and means to stop or recover. MEDAL++'s undo behavior and ConRFT's human intervention illustrate distinct recovery and oversight mechanisms with different reversibility limits [149, 30]. Redeployment then requires evidence for the new robot and scene beyond the training constraint or simulated damage test. The record should include rejected candidates, aborted trials, recovery outcomes, and the assumptions of each protective mechanism.

Constraint handling itself spans several assumptions. POLICEd RL constructs a closed-loop policy with hard-constraint guarantees for its specified constraint class; model-based actor–critic exploration with safety filters studies constrained data acquisition; and latent-space reachability extends safety assessment beyond hand-written collision constraints [20, 3, 127]. Gameplay filters use adversarial imagination to screen actions for complex robot dynamics [129]. These methods make a safety envelope more explicit, but their guarantees or empirical tests are tied to the modeled dynamics,

observations, and constraint definitions. An RSI process that changes any of those objects would need to re-establish the envelope before deploying the next version.

A stable protection interface for an adaptive learner. ENPIRE's human-assisted environment setup is instructive: agentic code edits occur inside a task-specific reset, verification and safety interface that is then held fixed during policy search [181]. AutoRT likewise couples automated task proposal with external safety rules and human oversight [8]. These arrangements define the editable decisions and reusable protection services. A system may propose a new training verifier, but that proposal is different from permission to weaken its own deployment gate. Candidate code, data and verifier versions should be auditable and reversible; the acceptance authority should be independent of the branch under test. Its criteria need not be frozen forever as hardware and tasks change, but a revision requires separate governance and bridge tests so that apparent improvement is not just a more permissive ruler. Statistical limits for self-modification characterize assumption-specific constraints on permitted edits [169]. Reporting human approvals, resets and emergency stops is more informative than a single "autonomous" label [123].

Distribution shift is particularly dangerous when a learned safety model is treated as a fixed oracle. Disturbance recasting adapts value-function safety filters when mismatch assumptions are incomplete; uncertainty-aware latent filters address overconfidence on out-of-distribution hazards; and multimodal reasoning can select safe fallback strategies before a runtime monitor triggers them [163, 146, 51]. These routes address model mismatch through filter adaptation, uncertainty detection, and fallback planning. The external gate should log which filter version intervened, what uncertainty it detected, and whether an apparently safe rollout was later found to violate a physical constraint.

Safety outcomes also depend on the task domain and recovery mechanism. Agile quadruped locomotion pairs fast control with a recovery policy; conformal uncertainty in crowd navigation modulates behavior under pedestrian shift; and tactile calibration addresses forces that a vision-only manipulation assessment may miss [64, 193, 177]. Proactive task-graph recovery in manipulation concerns the timing of failure response, while RoboChemist illustrates a long-horizon robot workflow in which hazardous materials and protocol compliance matter beyond simple task success [186, 207]. Feasibility-guided safety-critical scenario generation in autonomous driving warns that a severe adversarial test is not informative if it makes failure unavoidable rather than revealing a controllable weakness [27]. The resulting safety record should separate recoverability, force and damage, unavoidable hazards, and the scope of the external veto instead of compressing them into task success.

Exploration, deployment, and process-edit authority together define a trusted operating envelope. Within that envelope, a robot can learn increasingly effective experiment and repair strategies while retaining reliable fallback behavior. This modular separation is a practical enabler of process learning: it concentrates adaptation where experience is informative and keeps physical protection reusable as the learner changes.

7 Toward Recursive Self-Improving Physical AI

The preceding chapters identify a common research problem: learning which intervention will improve a particular robot under its available practice conditions. We organize the agenda around three tractable decisions—diagnosis-to-repair, experiment selection, and feedback or operator selection—and then consider how the resulting improvement knowledge could transfer. The five-level roadmap locates these decisions within a progression from human-directed updating to transferable process efficacy.

7.1 Missing Capabilities for Physical RSI

The central missing mechanism is *credit assignment to improvement decisions*. Action learning evaluates what a robot did in a task. Process learning evaluates whether a choice of experience, feedback, or update helped a subsequent learner. Its target is delayed, depends on the starting system, and consumes the same hardware resources as task practice. A useful training record must therefore connect a diagnosed weakness and chosen intervention to the capabilities retained after the resulting learning cycle.

Learn a diagnosis-to-repair mapping. Success detection, AHA's failure explanations, and FOREWARN's predicted consequences provide different descriptions of a weak rollout [44, 45, 180].

ReSteer identifies weakly steerable states, while LIBERO reveals forgetting across tasks [33, 105]. An explanation becomes actionable when it discriminates among interventions. For example, changing the action policy, updating a scene graph, and revising a body model address different sources of a failed grasp; interaction-derived memory and damage-adaptation studies supply concrete starting points for these alternatives [81, 18, 39]. A proposed repair selector could condition on trajectory evidence, learner state, and hardware condition, and learn from the subsequent gain and cost of the chosen intervention. Its prediction should be evaluated against competing repairs on later failures. This would test whether diagnostic features support useful decisions, beyond their accuracy in describing the original episode.

Select experiments for their downstream learning value. Task proposals, practice continuation, and data delivery are already available through fleet scheduling, SOAR's collection loop, learned undo behavior, and asynchronous infrastructure [8, 212, 149, 198]. SERL and LeRobot make experiments easier to execute, while autonomous-collection analyses identify preparation, reset, and outcome detection as continuing costs [109, 25, 123]. Adaptive curricula supply useful selection signals: reachable challenge, regret, and learning potential; task and reward generation expand the candidate pool [50, 41, 82, 175, 114]. A physical experiment selector must connect these signals to the update they enable. A difficult trial may have little value if its outcome is ambiguous or its terminal state prevents further practice. A useful proposed objective is the later retained gain obtainable from the selected experiment under the full robot, compute, and human budget. Learning this relationship would let a selector revise its priorities when coverage, recovery, or feedback becomes the active bottleneck.

Choose feedback and an update operator jointly. Visual verification can screen an action and generate later training trajectories; a frozen earlier checkpoint instead supplies a stable practice signal [203, 147]. Learned rewards extend the available supervision, while proxy analyses explain why optimizing a score can diverge from the intended outcome [90, 60, 153, 55]. Damage assessment, runtime protection, and probabilistic safety evaluation contribute measurements with different consequence and timing scopes [15, 6, 78]. The research question is which combination supplies enough information for a particular repair. Additional observation or human judgment is valuable when it changes an intervention choice; dense feedback is valuable when the learner can use it without amplifying a systematic labeling error.

The update alternatives differ in their information requirements. Broad retraining reuses accumulated experience, local reinforcement learning exploits deployment feedback, hindsight relabeling constructs goal supervision, distillation transfers a specialist's behavior, and imagination supplies model-dependent trajectories [19, 212, 121, 157, 211, 184, 191]. A scene-memory update may resolve a representation error with a smaller change than policy retraining; a mislabeled outcome may instead require repairing the feedback channel [81, 45]. A selector should learn the consequences of these choices, including retention and repair cost. This makes feedback quality and update scope coupled design variables.

Meta-learning and learned optimizers offer starting points for training update behavior across tasks; algorithm and workflow search expose larger procedural spaces in digital settings [48, 9, 137, 71]. ENPIRE's preprint makes robot training code editable inside a human-assisted environment interface [181]. ASPIRE and SHAPER explore control-program, skill, or harness revision, with principal quantitative evidence in simulation or benchmarks [108, 173]. For a physical study, restricting search to a named update component would make its cost and consequences easier to identify. Broader code search could follow once the value of simpler operator choices is established.

Accumulate improvement knowledge across generations. Workflow revision exposes candidate decisions, and statistical self-modification analyses clarify why edit assumptions matter [201, 169]. Procedural generalization tests and robot retention studies contribute complementary measurements [38, 105, 106]. The proposed learning target is intervention value conditioned on weakness, learner state, and operating conditions. Prospective comparisons against a predecessor process can supply training evidence for this model while controlling starting capability and opportunity. The model should retain knowledge of unsuccessful repairs as well as useful ones, so that later choices benefit from more than the latest accepted code change.

7.2 Roadmap to Physical RSI

Autonomy-centered RSI roadmaps motivate distinguishing system contribution from the breadth of improvement authority [46]. Our physical roadmap adds practice integration, future learning benefit, and transfer as organizing dimensions. Table 7 relates the five capability-and-scope levels to four evidence gates. A process edit is a candidate for P4; repeated future benefit establishes its bounded value. P5 adds transfer beyond that original scope.

Table 7: Proposed capability-and-scope levels and their evidence requirements. P1–P5 describe what the system controls and where it is useful; G1–G4 describe evidence for retained gains, linked cycles, process revisions, and future benefit. Transfer at P5 extends the scope of the process comparison.

Level	Decision capability	Evidence and gate	Next technical challenge
P1: Human-directed	A person chooses and carries out the update	Physical capability and resource measurements characterize the starting workflow	Give the system a useful role in producing updates
P2: System-assisted	The system contributes experience, feedback, or an update proposal	G1: a traceable contribution to an independently accepted, retained X gain	Integrate recovery, feedback, and redeployment
P3: Closed-loop SI	A fixed Ψ repeatedly practices, updates, and redeploys	G2: linked cycles with retained gains and recorded physical and human dependencies	Learn which practice or repair decisions help later learning
P4: Bounded RSI	The system revises Ψ from its outcomes within a stated scope	G3 identifies a process revision; repeated G4 advantage at common B, K supports its future value	Accumulate and generalize intervention-value estimates
P5: Transferable efficacy	Improvement knowledge adapts to held-out tasks, embodiments, or domains	Prospective process benefit beyond the original P4 scope, with adaptation costs	Identify transferable failure features and decision structure

First study: learn practice selection over a stable learner. Adaptable starting policies and real-robot learning systems provide a substrate for comparing experiment choices [21, 22, 160, 87, 149, 67, 19, 147, 91]. A tractable initial study would keep the feedback model, update algorithm, and deployment gate fixed while learning to select practice tasks. Scheduling, autonomous practice, and synchronization supply the execution services [8, 212, 198]; sequential evaluation and retention mechanisms support checking the resulting capability changes [105, 34, 88, 107, 141]. The learned selector and its frozen predecessor should start from matched robot states and receive the same budget and opportunity horizon. Randomized hardware blocks and condition records address wear and calibration drift. The main result is later retained gain under those resources, accompanied by recovery and human effort. This isolates one process decision while using the evaluation design in Section 5.3.

Second study: learn a small repair portfolio. Adaptive curricula, task or reward proposals, and criticality estimates provide candidate experiments; editable training code enlarges the repair space [8, 50, 41, 82, 175, 172, 114, 63, 181]. We favor first comparing a small portfolio of well-understood repairs, such as additional demonstrations, local policy practice, and a representation update. The selector would receive a diagnosis and predict which repair has the greatest downstream value within the resource and safety envelope. Holding acceptance services stable allows disagreement between predicted and realized repair value to become a learning signal. Expansion to feedback and optimizer choice should follow evidence that the simpler portfolio can be selected reliably.

Third study: transfer the improvement strategy. Simulation resources and visual or dynamics variation can supply held-out conditions for developing transfer tests [128, 124, 58, 162, 133]. Physical comparisons across sites and consequence-specific tests then characterize the target setting [12, 15, 79, 6]. A P5 study would compare the transferred revised process with its frozen predecessor on held-out learning problems under matched starting robot capabilities and target-domain budgets. A standard fixed process and a controller trained anew provide additional baselines for the value of transferred knowledge; their adaptation and development costs should be accounted for. The transferable object is the relationship between failure features and repair value. A benefit could appear as better experiment selection even when the target action policy must be relearned. The study should identify which parts of that relationship survive changes of task, embodiment, or feedback interface.

Our near-term view is a *bounded meta-controller over reusable learning and safety interfaces*. Its first competence is choosing better experiments and repairs; its longer-term competence is transporting those choices across tasks and embodiments. This route connects current physical SI to RSI through a concrete learning target: the downstream value of an improvement decision.

8 Conclusion

Physical self-improvement has become practical in several robot-learning settings through the joint design of useful experience, recoverable interaction, and feedback [149, 121, 67, 157, 19, 212, 147]. Our review shows why progress depends on the whole learning loop: broader collection helps when preparation, feedback, and policy delivery remain effective [86, 8, 198, 123], while virtual proposals and imagined experience extend search under model and transfer assumptions [175, 114, 179, 191, 162, 133]. Physical learning therefore requires coordinating data value with the cost and feasibility of continued practice, while preserving earlier skills [105].

The synthesis centers on three coupled decisions: what to practice, how to interpret its outcomes, and what to change. Outcome and diagnostic signals support different interventions [44, 45, 180, 203, 90]; relabeling, distillation, and corrective interaction change the available policy supervision [211, 184, 30, 33]; representation and code updates expand the repair space [81, 18, 39, 181, 108, 173]. The key process-learning problem is credit assignment: which decision caused a useful later improvement, for which learner and physical conditions? Research on trainable update behavior, curricula, and procedural search establishes relevant starting points [53, 48, 9, 50, 41, 137, 71, 201]. Versioned intervention outcomes can connect these ideas to physical learning.

We favor bounded meta-control over reusable learning and safety interfaces as an initial route to physical RSI. Feedback design must account for proxy mismatch [60, 153], and physical protection must retain its stated task, dynamics, and authority assumptions [15, 6, 1, 169]. Learning one practice or repair decision first makes its downstream value easier to identify. Prospective comparisons with the frozen predecessor then test whether process knowledge improves later learning under common resources. The longer-term objective is to transfer this knowledge: a robot should approach a new task with better judgments about which experience, feedback, and intervention are worth their physical cost.

References

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *International conference on machine learning*, pages 22–31. Pmlr, 2017.

[2] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, volume 2024, pages 21246–21263, 2024.

[3] Ahmed Agha, Baris Kayalibay, Atanas Mirchev, Patrick van der Smagt, and Justin Bayer. Exploring under constraints with model-based actor-critic and safety filters. In *CoRL*, pages 1216–1230, 2024.

[4] Christopher Agia, Rohan Sinha, Jingyun Yang, Zi-ang Cao, Rika Antonova, Marco Pavone, and Jeannette Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. *arXiv preprint arXiv:2410.04640*, 2024.

[5] Christopher Agia, Rohan Sinha, Jingyun Yang, Rika Antonova, Marco Pavone, Haruki Nishimura, Masha Itkina, and Jeannette Bohg. Cupid: Curating data your robot loves with influence functions. *arXiv preprint arXiv:2506.19121*, 2025.

[6] Riad Ahmed and Momotaz Begum. Tail-safe: Task-agnostic safety monitoring for imitation learning policies. *arXiv preprint arXiv:2605.01195*, 2026.

[7] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

[8] Michael Ahn, Debidatta Dwibedi, Chelsea Finn, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Karol Hausman, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, et al. Autort: Embodied foundation models for large scale orchestration of robotic agents. *arXiv preprint arXiv:2401.12963*, 2024.

[9] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando De Freitas. Learning to learn by gradient descent by gradient descent. *Advances in neural information processing systems*, 29, 2016.

[10] Abrar Anwar, Rohan Gupta, and Jesse Thomason. Contrast sets for evaluating language-guided robot policies. *arXiv preprint arXiv:2406.13636*, 2024.

[11] Yarden As, Lenart Treven, Carmelo Sferrazza, Stelian Coros, and Andreas Krause. Act-safe: Active exploration with safety constraints for reinforcement learning. In *International Conference on Learning Representations*, volume 2025, pages 29752–29776, 2025.

[12] Pranav Atreya, Karl Pertsch, Tony Lee, Moo Jin Kim, Arhan Jain, Artur Kuramshin, Clemens Eppner, Cyrus Neary, Edward Hu, Fabio Ramos, et al. Roboarena: Distributed real-world evaluation of generalist robot policies. *arXiv preprint arXiv:2506.18123*, 2025.

[13] Tewodros W Ayalew, Xiao Zhang, Kevin Yuanbo Wu, Tianchong Jiang, Michael Maire, and Matthew R Walter. Progressor: A perceptually guided reward estimator with self-supervised online refinement. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10297–10306. IEEE, 2025.

[14] Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew, and Igor Mordatch. Emergent tool use from multi-agent autocurricula. *arXiv preprint arXiv:1909.07528*, 2019.

[15] Arnav Balaji, Arpit Bahety, Sriniket Ambatipudi, Daniel Lam, Junhong Xu, and Roberto Martín-Martín. Oopsieverse: A safety benchmark with damage-aware simulation for robot manipulation. *arXiv preprint arXiv:2606.31993*, 2026.

[16] Philip J Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, pages 1577–1594. PMLR, 2023.

[17] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 41–48, 2009.

[18] Josh Bongard, Victor Zykov, and Hod Lipson. Resilient machines through continuous self-modeling. *Science*, 314(5802):1118–1121, 2006.

[19] Konstantinos Bousmalis, Giulia Vezzani, Dushyant Rao, Coline Devin, Alex X Lee, Maria Bauzá, Todor Davchev, Yuxiang Zhou, Agrim Gupta, Akhil Raju, et al. Robocat: A self-improving generalist agent for robotic manipulation. *arXiv preprint arXiv:2306.11706*, 2023.

[20] Jean-Baptiste Bouvier, Kartik Nagpal, and Negar Mehr. Policed rl: Learning closed-loop robot control policies with provable satisfaction of hard constraints. *arXiv preprint arXiv:2403.13297*, 2024.

[21] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.

[22] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[23] Antonia Bronars, Younghyo Park, and Pulkit Agrawal. Tune to learn: How controller gains shape robot policy learning. *arXiv preprint arXiv:2604.02523*, 2026.

[24] Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Shenyuan Gao, Xindong He, Xuan Hu, Xu Huang, et al. Agibot world colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025.

[25] Remi Cadene, Simon Alibert, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Jade Choghari, Martino Russi, Caroline Pascal, Steven Palma, et al. Lerobot: An open-source library for end-to-end robot learning. In *International Conference on Learning Representations*, volume 2026, pages 122398–122417, 2026.

[26] Akshay L Chandra, Iman Nematollahi, Chenguang Huang, Tim Welschehold, Wolfram Burgard, and Abhinav Valada. Diwa: Diffusion policy adaptation with world models. *arXiv preprint arXiv:2508.03645*, 2025.

[27] Keyu Chen, Yuheng Lei, Hao Cheng, Haoran Wu, Wenchao Sun, and Sifa Zheng. Frea: Feasibility-guided generation of safety-critical scenarios with reasonable adversariality. *arXiv preprint arXiv:2406.02983*, 2024.

[28] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in ai: From bounded self-refinement to autonomous research loops. *arXiv preprint arXiv:2607.07663*, 2026.

[29] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[30] Yuhui Chen, Shuai Tian, Shugao Liu, Yingting Zhou, Haoran Li, and Dongbin Zhao. Conrft: A reinforced fine-tuning method for vla models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025.

[31] Yujia Chen, Zaid Mahboob, and Bowen Weng. Betting for sim-to-real performance evaluation. In *Robotics: Science and Systems (RSS)*, 2026.

[32] Yutian Chen, Matthew W Hoffman, Sergio Gómez Colmenarejo, Misha Denil, Timothy P Lillicrap, Matt Botvinick, and Nando Freitas. Learning to learn without gradient descent by gradient descent. In *International Conference on Machine Learning*, pages 748–756. PMLR, 2017.

[33] Zhenyang Chen, Alan Tian, Liquan Wang, Benjamin Joffe, Yingyan Celine Lin, Yuxiao Chen, Siddharth Karamcheti, and Danfei Xu. Resteer: Quantifying and refining the steerability of multitask robot policies. *arXiv preprint arXiv:2603.17300*, 2026.

[34] Shuo Cheng, Zhaoyi Li, Kelin Yu, and Danfei Xu. Continual robot learning via language-guided skill acquisition. *Transactions on Machine Learning Research*, 2025.

[35] Cheng Chi, Zhenjia Xu, Chuer Pan, Eric Cousineau, Benjamin Burchfiel, Siyuan Feng, Russ Tedrake, and Shuran Song. Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots. *arXiv preprint arXiv:2402.10329*, 2024.

[36] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

[37] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.

[38] Karl Cobbe, Chris Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International conference on machine learning*, pages 2048–2056. PMLR, 2020.

[39] Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. Robots that can adapt like animals. *Nature*, 521(7553):503–507, 2015.

[40] Sudeep Dasari, Frederik Ebert, Stephen Tian, Suraj Nair, Bernadette Bucher, Karl Schmeckpeper, Siddharth Singh, Sergey Levine, and Chelsea Finn. Robonet: Large-scale multi-robot learning. *arXiv preprint arXiv:1910.11215*, 2019.

[41] Michael Dennis, Natasha Jaques, Eugene Vinitsky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. *Advances in neural information processing systems*, 33:13049–13061, 2020.

[42] Jingtao Ding, Yunke Zhang, Yu Shang, Yuheng Zhang, Zefang Zong, Jie Feng, Yuan Yuan, Hongyuan Su, Nian Li, Nicholas Sukiennik, et al. Understanding world or predicting future? a comprehensive survey of world models. *ACM Computing Surveys*, 58(3):1–38, 2025.

[43] Lakshita Dodeja, Ondrej Biza, Shivam Vats, Stephen Hart, Stefanie Tellex, Robin Walters, Karl Schmeckpeper, and Thomas Weng. When life gives you bc, make q-functions: Extracting q-values from behavior cloning for on-robot reinforcement learning. *arXiv preprint arXiv:2605.05172*, 2026.

[44] Yuqing Du, Ksenia Konyushkova, Misha Denil, Akhil Raju, Jessica Landon, Felix Hill, Nando De Freitas, and Serkan Cabi. Vision-language models as success detectors. *arXiv preprint arXiv:2303.07280*, 2023.

[45] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *arXiv preprint arXiv:2410.00371*, 2024.

[46] Yi Duan, Ying Liu, Zirui Tang, Haodong Chen, Jun Zhou, Yumou Liu, Bangrui Xu, Yukai Wu, Sidi Chen, Yuhan Zhou, et al. The last ai built by humans: Toward genuine recursive self-improvement. *arXiv preprint arXiv:2609.11873*, 2026.

[47] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.

[48] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.

[49] Chelsea Finn, Tianhe Yu, Tianhao Zhang, Pieter Abbeel, and Sergey Levine. One-shot visual imitation learning via meta-learning. In *Conference on robot learning*, pages 357–368. PMLR, 2017.

[50] Carlos Florensa, David Held, Xinyang Geng, and Pieter Abbeel. Automatic goal generation for reinforcement learning agents. In *International conference on machine learning*, pages 1515–1528. PMLR, 2018.

[51] Milan Ganai, Rohan Sinha, Christopher Agia, Daniel Morton, Luigi Di Lillo, and Marco Pavone. Real-time out-of-distribution failure prevention via multi-modal reasoning. *arXiv preprint arXiv:2505.10547*, 2025.

[52] Milan Ganai, Katie Luo, Jonas Frey, Clark Barrett, and Marco Pavone. Self-supervised bootstrapping of action-predictive embodied reasoning. *arXiv preprint arXiv:2602.08167*, 2026.

[53] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, et al. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *arXiv preprint arXiv:2507.21046*, 2025.

[54] Jensen Gao, Annie Xie, Ted Xiao, Chelsea Finn, and Dorsa Sadigh. Efficient data collection for robotic manipulation via compositional generalization. *arXiv preprint arXiv:2403.05110*, 2024.

[55] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International conference on machine learning*, pages 10835–10866. PMLR, 2023.

[56] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Nan Duan, Weizhu Chen, et al. Critic: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, volume 2024, pages 57734–57811, 2024.

[57] Luca Grillotti, Lisa Coiffard, Oscar Pang, Maxence Faldor, and Antoine Cully. From tabula rasa to emergent abilities: Discovering robot skills via real-world unsupervised quality-diversity. *arXiv preprint arXiv:2508.19172*, 2025.

[58] Jiayuan Gu, Fanbo Xiang, Xuanlin Li, Zhan Ling, Xiqiang Liu, Tongzhou Mu, Yihe Tang, Stone Tao, Xinyue Wei, Yunchao Yao, et al. Maniskill2: A unified benchmark for generalizable manipulation skills. *arXiv preprint arXiv:2302.04659*, 2023.

[59] Sehoon Ha, Peng Xu, Zhenyu Tan, Sergey Levine, and Jie Tan. Learning to walk in the real world with minimal human effort. *arXiv preprint arXiv:2002.08550*, 2020.

[60] Dylan Hadfield-Menell, Smitha Milli, Pieter Abbeel, Stuart J Russell, and Anca Dragan. Inverse reward design. *Advances in neural information processing systems*, 30, 2017.

[61] Asher Hancock, Xindi Wu, Lihan Zha, Olga Russakovsky, and Anirudha Majumdar. Actions as language: Fine-tuning vlms into vlas without catastrophic forgetting. In *International Conference on Learning Representations*, volume 2026, pages 75333–75360, 2026.

[62] Nick Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, volume 2024, pages 47376–47405, 2024.

[63] Linxuan He, Yuying Tian, Lingxiang Fan, Jiaqi Pi, Yinqiao Lu, Shang Su, Mengkai Shi, and Shuo Feng. Self-evolving learning for embodied ai with criticality model. *arXiv preprint arXiv:2607.28251*, 2026.

[64] Tairan He, Chong Zhang, Wenli Xiao, Guanqi He, Changliu Liu, and Guanya Shi. Agile but safe: Learning collision-free high-speed legged locomotion. *arXiv preprint arXiv:2401.17583*, 2024.

[65] Joey Hejna, Chethan Bhateja, Yichen Jiang, Karl Pertsch, and Dorsa Sadigh. Re-mix: Optimizing data mixtures for large scale imitation learning. *arXiv preprint arXiv:2408.14037*, 2024.

[66] Joey Hejna, Suvir Mirchandani, Ashwin Balakrishna, Annie Xie, Ayzaan Wahid, Jonathan Tompson, Pannag Sanketi, Dhruv Shah, Coline Devin, and Dorsa Sadigh. Robot data curation with mutual information estimators. *arXiv preprint arXiv:2502.08623*, 2025.

[67] Noriaki Hirose, Dhruv Shah, Kyle Stachowicz, Ajay Sridhar, and Sergey Levine. Selfi: Autonomous self-improvement with rl for vision-based navigation around people. In *8th Annual Conference on Robot Learning*, 2024.

[68] Hengyuan Hu, Suvir Mirchandani, and Dorsa Sadigh. Imitation bootstrapped reinforcement learning. *arXiv preprint arXiv:2311.02198*, 2023.

[69] Jiaheng Hu, Peter Stone, and Roberto Martín-Martín. Slac: Simulation-pretrained latent action space for whole-body real-world rl. In *9th Annual Conference on Robot Learning*, 2025.

[70] Kaizhe Hu, Haochen Shi, Yao He, Weizhuo Wang, C Karen Liu, and Shuran Song. Robot trains robot: Automatic real-world policy adaptation and learning for humanoids. *arXiv preprint arXiv:2508.12252*, 2025.

[71] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, volume 2025, pages 21344–21377, 2025.

[72] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.

[73] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023.

[74] Yixuan Huang, Novella Alvina, Mohanraj Devendran Shanthi, and Tucker Hermans. Fail2progress: Learning from real-world robot failures with stein variational inference. *arXiv preprint arXiv:2509.01746*, 2025.

[75] Yixuan Huang, Bowen Li, Vaibhav Saxena, Yichao Liang, Utkarsh Aashu Mishra, Liang Ji, Lihan Zha, Jimmy Wu, Nishanth Kumar, Sebastian Scherer, et al. Kinder: A physical reasoning benchmark for robot learning and planning. *arXiv preprint arXiv:2604.25788*, 2026.

[76] Arhan Jain, Mingtong Zhang, Kanav Arora, William Chen, Marcel Torne, Muhammad Zubair Irshad, Sergey Zakharov, Yue Wang, Sergey Levine, Chelsea Finn, et al. Polaris: Scalable real-to-sim evaluations for generalist robot policies. *arXiv preprint arXiv:2512.16881*, 2025.

[77] Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J Davison. Rlbench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 5(2): 3019–3026, 2020.

[78] Inkyu Jang, Jonghae Park, Sihyun Cho, Chams E Mballo, Claire J Tomlin, and H Jin Kim. Eigensafe: A spectral framework for learning-based probabilistic safety assessment. *arXiv preprint arXiv:2509.17750*, 2025.

[79] Jiaming Ji, Borong Zhang, Jiayi Zhou, Xuehai Pan, Weidong Huang, Ruiyang Sun, Yiran Geng, Yifan Zhong, Josef Dai, and Yaodong Yang. Safety gymnasium: A unified safe reinforcement learning benchmark. *Advances in Neural Information Processing Systems*, 36:18964–18993, 2023.

[80] Yufei Jia, Heng Zhang, Ziheng Zhang, Junzhe Wu, Mingrui Yu, Zifan Wang, Dixuan Jiang, Zheng Li, Chenyu Cao, Zhuoyuan Yu, et al. Gs-playground: A high-throughput photorealistic simulator for vision-informed robot learning. *arXiv preprint arXiv:2604.25459*, 2026.

[81] Hanxiao Jiang, Binghao Huang, Ruihai Wu, Zhuoran Li, Shubham Garg, Hooshang Nayyeri, Shenlong Wang, and Yunzhu Li. Roboexp: Action-conditioned scene graph via interactive exploration for robotic manipulation. *arXiv preprint arXiv:2402.15487*, 2024.

[82] Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning*, pages 4940–4950. PMLR, 2021.

[83] Yunfan Jiang, Chen Wang, Ruohan Zhang, Jiajun Wu, and Li Fei-Fei. Transic: Sim-to-real policy transfer by learning from online correction. *arXiv preprint arXiv:2405.10315*, 2024.

[84] Yunfan Jiang, Ruohan Zhang, Josiah Wong, Chen Wang, Yanjie Ze, Hang Yin, Cem Gokmen, Shuran Song, Jiajun Wu, and Li Fei-Fei. Behavior robot suite: Streamlining real-world whole-body manipulation for everyday household activities. *arXiv preprint arXiv:2503.05652*, 2025.

[85] Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, et al. Scalable deep reinforcement learning for vision-based robotic manipulation. In *Conference on robot learning*, pages 651–673. PMLR, 2018.

[86] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.

[87] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[88] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.

[89] Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. Rma: Rapid motor adaptation for legged robots. *arXiv preprint arXiv:2107.04034*, 2021.

[90] Tony Lee, Andrew Wagenmaker, Karl Pertsch, Percy Liang, Sergey Levine, and Chelsea Finn. Roboreward: General-purpose vision-language reward models for robotics. *arXiv preprint arXiv:2601.00675*, 2026.

[91] Kun Lei, Huanyu Li, Dongjie Yu, Zhenyu Wei, Lingxiao Guo, Zhennan Jiang, Ziyu Wang, Shiyu Liang, and Huazhe Xu. Performant robotic manipulation with real-world reinforcement learning. *Science Robotics*, 11(116):eaed6267, 2026.

[92] Jacob Levy, Tyler Westenbroek, and David Fridovich-Keil. Learning to walk from three minutes of real-world data with semi-structured dynamics models. *arXiv preprint arXiv:2410.09163*, 2024.

[93] Jacob Levy, Jason Gibson, Bogdan Vlahov, Erica Tevere, Evangelos Theodorou, David Fridovich-Keil, and Patrick Spieler. Meta-learning online dynamics model adaptation in off-road autonomous driving. *arXiv preprint arXiv:2504.16923*, 2025.

[94] Jacob Levy, Tyler Westenbroek, Kevin Huang, Fernando Palafox, Patrick Yin, Shayegan Omidshafiei, Dong-Ki Kim, Abhishek Gupta, and David Fridovich-Keil. Simulation distillation: Pretraining world models in simulation for rapid real-world adaptation. *arXiv preprint arXiv:2603.15759*, 2026.

[95] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Michael Lingelbach, Jiankai Sun, et al. Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning*, pages 80–93. PMLR, 2023.

[96] Puhao Li, Yingying Wu, Ziheng Xi, Wanlin Li, Yuzhe Huang, Zhiyuan Zhang, Yinghan Chen, Jianan Wang, Song-Chun Zhu, Tengyu Liu, et al. Controlvla: Few-shot object-centric adaptation for pre-trained vision-language-action models. *arXiv preprint arXiv:2506.16211*, 2025.

[97] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, et al. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.

[98] Zhaoting Li, Gang Chen, Javier Alonso-Mora, Cosimo Della Santina, and Jens Kober. Set-supervised diffusion policy: Learning action-chunking diffusion through corrections. *arXiv preprint arXiv:2606.01865*, 2026.

[99] Anthony Liang, Yigit Korkmaz, Jiahui Zhang, Minyoung Hwang, Abrar Anwar, Sidhant Kaushik, Aditya Shah, Alex S Huang, Luke Zettlemoyer, Dieter Fox, et al. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *arXiv preprint arXiv:2603.02115*, 2026.

[100] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*, pages 9493–9500. IEEE, 2023.

[101] Jacky Liang, Fei Xia, Wenhao Yu, Andy Zeng, Montserrat Gonzalez Arenas, Maria Attarian, Maria Bauza, Matthew Bennice, Alex Bewley, Adil Dostmohamed, et al. Learning to learn faster from human feedback with language model predictive control. *arXiv preprint arXiv:2402.11450*, 2024.

[102] Fanqi Lin, Kushal Arora, Jean Mercat, Haruki Nishimura, Paarth Shah, Chen Xu, Mengchao Zhang, Mark Zolotas, Maya Angeles, Owen Pfannenstiehl, et al. A systematic study of data modalities and strategies for co-training large behavior models for robot manipulation. *arXiv preprint arXiv:2602.01067*, 2026.

[103] Kevin Lin, Varun Ragunath, Andrew McAlinden, Aaditya Prasad, Jimmy Wu, Yuke Zhu, and Jeannette Bohg. Constraint-preserving data generation for one-shot visuomotor policy generalization. In *9th Annual Conference on Robot Learning*, 2025.

[104] Toru Lin, Kartik Sachdev, Linxi Fan, Jitendra Malik, and Yuke Zhu. Sim-to-real reinforcement learning for vision-based dexterous manipulation on humanoids. *arXiv preprint arXiv:2502.20396*, 2025.

[105] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.

[106] Yuan Liu, Haoran Li, Shuai Tian, Yuxing Qin, Yuhui Chen, Yupeng Zheng, Yongzhen Huang, and Dongbin Zhao. Towards long-lived robots: Continual learning vla models via reinforcement fine-tuning. *arXiv preprint arXiv:2602.10503*, 2026.

[107] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. *Advances in neural information processing systems*, 30, 2017.

[108] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, et al. Aspire: Agentic/skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.

[109] Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. Serl: A software suite for sample-efficient robotic reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 16961–16969. IEEE, 2024.

[110] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105): eads5033, 2025.

[111] Corey Lynch, Mohi Khansari, Ted Xiao, Vikash Kumar, Jonathan Tompson, Sergey Levine, and Pierre Sermanet. Learning latent plans from play. In *Conference on robot learning*, pages 1113–1132. Pmlr, 2020.

[112] Yecheng Jason Ma, Shagun Sodhani, Dinesh Jayaraman, Osbert Bastani, Vikash Kumar, and Amy Zhang. Vip: Towards universal visual reward and representation via value-implicit pre-training. *arXiv preprint arXiv:2210.00030*, 2022.

[113] Yecheng Jason Ma, Vikash Kumar, Amy Zhang, Osbert Bastani, and Dinesh Jayaraman. Liv: Language-image representations and rewards for robotic control. In *International Conference on Machine Learning*, pages 23301–23320. PMLR, 2023.

[114] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Jim Fan, et al. Eureka: Human-level reward design via coding large language models. In *International conference on learning Representations*, volume 2024, pages 26516–26560, 2024.

[115] Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. A survey on vision–language–action models for embodied ai. *IEEE Transactions on Neural Networks and Learning Systems*, 2026.

[116] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.

[117] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. *arXiv preprint arXiv:2108.03298*, 2021.

[118] Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. *arXiv preprint arXiv:2310.17596*, 2023.

[119] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.

[120] Shaunak A Mehta, Ananya Hazarika, Haochen Zhang, Fan Yang, Ryo Moriyama, Wenkai Li, Yash Patel, and Kanata Suzuki. Toward unified robot learning: Bridging representation, vision-language-action, and world models. *arXiv preprint arXiv:2609.03927*, 2026.

[121] Russell Mendonca, Emmanuel Panov, Bernadette Bucher, Jiuguang Wang, and Deepak Pathak. Continuously improving mobile manipulation with autonomous real-world rl. *arXiv preprint arXiv:2409.20568*, 2024.

[122] Helena Merker, Nick Walker, and Andreea Bobu. Robots that know what to ask: Recovering misaligned rewards through targeted explanations. *arXiv preprint arXiv:2605.22986*, 2026.

[123] Suvir Mirchandani, Suneel Belkhale, Joey Hejna, Evelyn Choi, Md Sazzad Islam, and Dorsa Sadigh. So you think you can scale up autonomous robot data collection? *arXiv preprint arXiv:2411.01813*, 2024.

[124] Yao Mu, Tianxing Chen, Zanxin Chen, Shijia Peng, Zhiqian Lan, Zeyu Gao, Zhixuan Liang, Qiaojun Yu, Yude Zou, Mingkun Xu, et al. Robotwin: Dual-arm robot benchmark with generative digital twins. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 27649–27660. IEEE, 2025.

[125] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3m: A universal visual representation for robot manipulation. *arXiv preprint arXiv:2203.12601*, 2022.

[126] Mitsuhiko Nakamoto, Simon Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. *Advances in Neural Information Processing Systems*, 36:62244–62269, 2023.

[127] Kensuke Nakamura, Lasse Peters, and Andrea Bajcsy. Generalizing safety beyond collision-avoidance via latent-space reachability analysis. *arXiv preprint arXiv:2502.00935*, 2025.

[128] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of household tasks for generalist robots. 2024.

[129] Duy P Nguyen, Kai-Chieh Hsu, Wenhao Yu, Jie Tan, and Jaime F Fisac. Gameplay filters: Robust zero-shot safety through adversarial imagination. *arXiv preprint arXiv:2405.00846*, 2024.

[130] Pablo Ortega-Kral, Eliot Xing, Arthur Bucker, Vernon Luk, Junseo Kim, Owen Kwon, Angchen Xie, Nikhil Sobanbabu, Yifu Yuan, Megan Lee, et al. Rio: Flexible real-time robot i/o for cross-embodiment robot learning. *arXiv preprint arXiv:2605.11564*, 2026.

[131] Georgios Papagiannis and Edward Johns. Miles: Making imitation learning easy with self-supervision. *arXiv preprint arXiv:2410.19693*, 2024.

[132] Anjali Parashar, Joseph Zhang, Yingke Li, and Chuchu Fan. Cost-aware discovery of contextual failures using bayesian active learning. In *9th Annual Conference on Robot Learning*, 2025.

[133] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 3803–3810. IEEE, 2018.

[134] Aggelos Psiris, Vasileios Argyriou, Evangelos K Markakis, Panagiotis Sarigiannidis, Efstratios Gavves, Kostas Bekris, Arash Ajoudani, and Georgios Th Papadopoulos. Foundation models in robotics: A comprehensive review of methods, models, datasets, challenges and future research directions. *arXiv preprint arXiv:2604.15395*, 2026.

[135] Wilbert Pumacay, Ishika Singh, Jiafei Duan, Ranjay Krishna, Jesse Thomason, and Dieter Fox. The colosseum: A benchmark for evaluating generalization for robotic manipulation. *arXiv preprint arXiv:2402.08191*, 2024.

[136] Ryan Punamiya, Simar Kareer, Zeyi Liu, Josh Citron, Ri-Zhao Qiu, Xiongyi Cai, Alexey Gavryushin, Jiaqi Chen, Davide Liconti, Lawrence Y Zhu, et al. Egoverse: An egocentric human dataset for robot learning from around the world. *arXiv preprint arXiv:2604.07607*, 2026.

[137] Esteban Real, Chen Liang, David So, and Quoc Le. Automl-zero: Evolving machine learning algorithms from scratch. In *International conference on machine learning*, pages 8007–8019. Pmlr, 2020.

[138] Kejia Ren, Gaotian Wang, Andrew S Morgan, and Kaiyu Hang. Zero-shot sim-to-real robot learning: A dexterous manipulation study on reactive catching. *arXiv preprint arXiv:2605.09789*, 2026.

[139] Yunfan Ren, Zhiyuan Zhu, Jiaxu Xing, and Davide Scaramuzza. Learning agile quadrotor flight in the real world. *arXiv preprint arXiv:2602.10111*, 2026.

[140] Zhe Ren, Yimeng Chen, Dandan Guo, Guowei Rong, Tonghui Li, RB Xiong, Qingfeng Lan, Wenyi Wang, Li Nanbo, Yibo Yang, et al. Self-improvements in modern agentic systems: A survey. *arXiv preprint arXiv:2607.13104*, 2026.

[141] David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. *Advances in neural information processing systems*, 32, 2019.

[142] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.

[143] Jürgen Schmidhuber. Godel machines: Self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint cs.LO/0309048*, 2003.

[144] Clemens Schwarke, Victor Klemm, Joshua Bagajo, Jean-Pierre Sleiman, Ignat Georgiev, Jesus Tordesillas, and Marco Hutter. Learning deployable locomotion control via differentiable simulation. *arXiv preprint arXiv:2404.02887*, 2024.

[145] Gershom Seneviratne, Jianyu An, Sahire Ellahy, Kasun Weerakoon, Mohamed Bashir Elnoor, Jonathan Deepak Kannan, Amogha Thalihalla Sunil, and Dinesh Manocha. Halo: Human preference aligned offline reward learning for robot navigation. *arXiv preprint arXiv:2508.01539*, 2025.

[146] Junwon Seo, Kensuke Nakamura, and Andrea Bajcsy. Uncertainty-aware latent safety filters for avoiding out-of-distribution failures. *arXiv preprint arXiv:2505.00779*, 2025.

[147] Seyed Kamyar Seyed Ghasemipour, Ayzaan Wahid, Jonathan Tompson, Pannag Sanketi, and Igor Mordatch. Self-improving embodied foundation models. *Advances in Neural Information Processing Systems*, 38:112651–112686, 2026.

[148] Carmelo Sferrazza, Dun-Ming Huang, Xingyu Lin, Youngwoon Lee, and Pieter Abbeel. Humanoidbench: Simulated humanoid benchmark for whole-body locomotion and manipulation. *arXiv preprint arXiv:2403.10506*, 2024.

[149] Archit Sharma, Ahmed M Ahmed, Rehaan Ahmad, and Chelsea Finn. Self-improving robots: End-to-end autonomous visuomotor reinforcement learning. *arXiv preprint arXiv:2303.01488*, 2023.

[150] Fan Shi, Chong Zhang, Takahiro Miki, Joonho Lee, Marco Hutter, and Stelian Coros. Rethinking robustness assessment: Adversarial attacks on learning-based quadrupedal locomotion controllers. *arXiv preprint arXiv:2405.12424*, 2024.

[151] Lucy Xiaoyang Shi, Zheyuan Hu, Tony Z Zhao, Archit Sharma, Karl Pertsch, Jianlan Luo, Sergey Levine, and Chelsea Finn. Yell at your robot: Improving on-the-fly from language corrections. *arXiv preprint arXiv:2403.12910*, 2024.

[152] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.

[153] Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. *Advances in neural information processing systems*, 35:9460–9471, 2022.

[154] David Snyder, Asher James Hancock, Apurva Badithela, Emma Dixon, Patrick Miller, Rares Andrei Ambrus, Anirudha Majumdar, Masha Itkina, and Haruki Nishimura. Is your imitation learning policy better than mine? policy comparison with near-optimal stopping. *arXiv preprint arXiv:2503.10966*, 2025.

[155] Alexander Spiridonov, Jan-Nico Zaech, Nikolay Nikolov, Luc Van Gool, and Danda Pani Paudel. Generalist robot manipulation beyond action labeled data. *arXiv preprint arXiv:2509.19958*, 2025.

[156] Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. Ricl: Adding in-context adaptability to pre-trained vision-language-action models. *arXiv preprint arXiv:2508.02062*, 2025.

[157] Kyle Stachowicz, Lydia Ignatova, and Sergey Levine. Lifelong autonomous improvement of navigation foundation models in the wild. In *Conference on Robot Learning*, pages 1035–1047. PMLR, 2025.

[158] Luke Strgar, David Matthews, Tyler Hummer, and Sam Kriegman. Evolution and learning in differentiable robots. *arXiv preprint arXiv:2405.14712*, 2024.

[159] Sainbayar Sukhbaatar, Zeming Lin, Ilya Kostrikov, Gabriel Synnaeve, Arthur Szlam, and Rob Fergus. Intrinsic motivation and automatic curricula via asymmetric self-play. *arXiv preprint arXiv:1703.05407*, 2017.

[160] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[161] Brijen Thananjeyan, Ashwin Balakrishna, Suraj Nair, Michael Luo, Krishnan Srinivasan, Minho Hwang, Joseph E Gonzalez, Julian Ibarz, Chelsea Finn, and Ken Goldberg. Recovery rl: Safe reinforcement learning with learned recovery zones. *IEEE Robotics and Automation Letters*, 6(3):4915–4922, 2021.

[162] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 23–30. IEEE, 2017.

[163] Sander Tonkens, Nikhil Uday Shinde, Azra Begzadić, Michael C Yip, Jorge Cortés, and Sylvia L Herbert. From space to time: Enabling adaptive safety with learned value functions via disturbance recasting. *arXiv preprint arXiv:2509.19597*, 2025.

[164] Marcel Torne, Arhan Jain, Jiayi Yuan, Vidaaranya Macha, Lars Ankile, Anthony Simeonov, Pulkit Agrawal, and Abhishek Gupta. Robot learning with super-linear scaling. *arXiv preprint arXiv:2412.01770*, 2024.

[165] Shivam Vats, Michelle Zhao, Patrick Callaghan, Mingxi Jia, Maxim Likhachev, Oliver Kroemer, and George Konidaris. Optimal interactive learning on the job via facility location planning. *arXiv preprint arXiv:2505.00490*, 2025.

[166] Akifumi Wachi, Wataru Hashimoto, Xun Shen, and Kazumune Hashimoto. Safe exploration in reinforcement learning: A generalized formulation and algorithms. *Advances in Neural Information Processing Systems*, 36:29252–29272, 2023.

[167] Homer Rich Walke, Kevin Black, Tony Z Zhao, Quan Vuong, Chongyi Zheng, Philippe Hansen-Estruch, Andre Wang He, Vivek Myers, Moo Jin Kim, Max Du, et al. Bridgedata v2: A dataset for robot learning at scale. In *Conference on robot learning*, pages 1723–1736. PMLR, 2023.

[168] Weikang Wan, Jiawei Fu, Xiaodi Yuan, Yifeng Zhu, and Hao Su. Lodestar: Long-horizon dexterity via synthetic data augmentation from human demonstrations. In *9th Annual Conference on Robot Learning*, 2025.

[169] Charles L Wang, Keir Dorchen, and Peter Jin. On the statistical limits of self-improving agents. *arXiv preprint arXiv:2510.04399*, 2025.

[170] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[171] Hao Wang, Joshua Bowden, Colton Crosby, and Somil Bansal. Offline policy evaluation for manipulation policies via discounted liveness formulation. *arXiv preprint arXiv:2605.11479*, 2026.

[172] Lirui Wang, Yiyang Ling, Zhecheng Yuan, Mohit Shridhar, Chen Bao, Yuzhe Qin, Bailin Wang, Huazhe Xu, and Xiaolong Wang. Gensim: Generating robotic simulation tasks via large language models. In *International Conference on Learning Representations*, volume 2024, pages 4890–4924, 2024.

[173] Peidong Wang, Zhiming Ma, Ying Chang, Xufang Luo, Xiaocui Yang, Shi Feng, Yuqing Yang, and Dongsheng Li. Self-evolving embodied agents via skill-harness evolution. *arXiv preprint arXiv:2608.11350*, 2026.

[174] Yixuan Wang, Rhythm Syed, Fangyu Wu, Mengchao Zhang, Aykut Onol, Jose Barreiros, Hooshang Nayyeri, Tony Dear, Huan Zhang, and Yunzhu Li. Interactive world simulator for robot policy training and evaluation. *arXiv preprint arXiv:2603.08546*, 2026.

[175] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. Robogen: Towards unleashing infinite data for automated robot learning via generative simulation. *arXiv preprint arXiv:2311.01455*, 2023.

[176] Ziyao Wang, Bingying Wang, Hanrong Zhang, Tingting Du, Tianyang Chen, Guoheng Sun, Yexiao He, Zheyu Shen, Wanghao Ye, and Ang Li. Vision-language-action in robotics: A survey of datasets, benchmarks, and data engines. *arXiv preprint arXiv:2604.23001*, 2026.

[177] Lai Wei, Jiahua Ma, Yibo Hu, and Ruimao Zhang. Ensuring force safety in vision-guided robotic manipulation via implicit tactile calibration. *arXiv preprint arXiv:2412.10349*, 2024.

[178] Kun Wu, Chengkai Hou, Jiaming Liu, Zhengping Che, Xiaozhu Ju, Zhuqin Yang, Meng Li, Yinuo Zhao, Zhiyuan Xu, Guang Yang, et al. Robomind: Benchmark on multi-embodiment intelligence normative data for robot manipulation. *arXiv preprint arXiv:2412.13877*, 2024.

[179] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Pieter Abbeel, and Ken Goldberg. Daydreamer: World models for physical robot learning. In *Conference on robot learning*, pages 2226–2240. PMLR, 2023.

[180] Yilin Wu, Ran Tian, Gokul Swamy, and Andrea Bajcsy. From foresight to forethought: Vlm-in-the-loop policy steering via latent alignment. *arXiv preprint arXiv:2502.01828*, 2025.

[181] Wenli Xiao, Jia Xie, Tonghe Zhang, Haotian Lin, Letian Fu, Haoru Xue, Jalen Lu, Yi Yang, Cunxi Dai, Zi Wang, et al. Enpire: Agentic robot policy self-improvement in the real world. *arXiv preprint arXiv:2606.19980*, 2026.

[182] Amber Xie, Rahul Chand, Dorsa Sadigh, and Joey Hejna. Data retrieval with importance weights for few-shot imitation learning. *arXiv preprint arXiv:2509.01657*, 2025.

[183] Youguang Xing, Xu Luo, Junlin Xie, Lianli Gao, Hengtao Shen, and Jingkuan Song. Shortcut learning in generalist robot policies: The role of dataset diversity and fragmentation. *arXiv preprint arXiv:2508.06426*, 2025.

[184] Charles Xu, Qiyang Li, Jianlan Luo, and Sergey Levine. Rldg: Robotic generalist policy distillation via reinforcement learning. *arXiv preprint arXiv:2412.09858*, 2024.

[185] Chen Xu, Tony Khuong Nguyen, Emma Dixon, Christopher Rodriguez, Patrick Miller, Robert Lee, Paarth Shah, Rares Ambrus, Haruki Nishimura, and Masha Itkina. Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies. *arXiv preprint arXiv:2503.08558*, 2025.

[186] Sheng Xu, Ruixing Jin, Huayi Zhou, Bo Yue, Guanren Qiao, Yunxin Tai, Yueci Deng, Kui Jia, and Guiliang Liu. From reaction to anticipation: Proactive failure recovery through agentic task graph for robotic manipulation. *arXiv preprint arXiv:2605.11951*, 2026.

[187] Xiuwei Xu, Angyuan Ma, Hankun Li, Bingyao Yu, Zheng Zhu, Jie Zhou, and Jiwen Lu. R2rgen: Real-to-real 3d data generation for spatially generalized manipulation. *arXiv preprint arXiv:2510.08547*, 2025.

[188] Zhengrong Xue, Shuying Deng, Zhenyang Chen, Yixuan Wang, Zhecheng Yuan, and Huazhe Xu. Demogen: Synthetic demonstration generation for data-efficient visuomotor policy learning. *arXiv preprint arXiv:2502.16932*, 2025.

[189] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, volume 2024, pages 12028–12068, 2024.

[190] Jenai Xuning Yang, Rishit Dagli, Alex Zook, Hugo Hadfield, Ankit Goyal, Stan Birchfield, Fabio Ramos, and Jonathan Tremblay. Robolab: A high-fidelity simulation benchmark for analysis of task generalist policies. *arXiv preprint arXiv:2604.09860*, 2026.

[191] Jiazhi Yang, Kunyang Lin, Jinwei Li, Wencong Zhang, Tianwei Lin, Longyan Wu, Zhizhong Su, Hao Zhao, Ya-Qin Zhang, Li Chen, et al. Rise: Self-improving robot policy with compositional world model. *arXiv preprint arXiv:2602.11075*, 2026.

[192] Jonathan Yang, Catherine Glossop, Arjun Bhorkar, Dhruv Shah, Quan Vuong, Chelsea Finn, Dorsa Sadigh, and Sergey Levine. Pushing the limits of cross-embodiment learning for manipulation and navigation. *arXiv preprint arXiv:2402.19432*, 2024.

[193] Jianpeng Yao, Xiaopan Zhang, Yu Xia, Zejin Wang, Amit K Roy-Chowdhury, and Jiachen Li. Towards generalizable safety in crowd navigation via conformal uncertainty handling. *arXiv preprint arXiv:2508.05634*, 2025.

[194] Justin Yu, Letian Fu, Huang Huang, Karim El-Refai, Rares Andrei Ambrus, Richard Cheng, Muhammad Zubair Irshad, and Ken Goldberg. Real2render2real: Scaling robot data without dynamics simulation or robot hardware. *arXiv preprint arXiv:2505.09601*, 2025.

[195] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on robot learning*, pages 1094–1100. PMLR, 2020.

[196] Youwei Yu, Jionghao Wang, Zhengming Yu, Wenping Wang, and Lantao Liu. Learning what matters: Adaptive information-theoretic objectives for robot exploration. *arXiv preprint arXiv:2605.12084*, 2026.

[197] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

[198] Hongzhi Zang, S Yu, H Lin, T Zhou, Z Huang, Z Guo, X Xu, Y Sheng, J Zhou, S Zhang, et al. User: A unified and extensible system for online real-world policy learning in embodied ai. In *Robotics: Science and Systems (RSS)*, 2026.

[199] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. *arXiv preprint arXiv:2403.03954*, 2024.

[200] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.

[201] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: open-ended evolution of self-improving agents. In *International Conference on Learning Representations*, volume 2026, pages 104223–104294, 2026.

[202] Jiahui Zhang, Yusen Luo, Abrar Anwar, Sumedh Anand Sontakke, Joseph J Lim, Jesse Thomason, Erdem Biyik, and Jesse Zhang. Rewind: Language-guided rewards teach robot policies without new demonstrations. *arXiv preprint arXiv:2505.10911*, 2025.

[203] Mingtong Zhang and Dhruv Shah. Visual verification enables inference-time steering and autonomous policy improvement. *arXiv preprint arXiv:2606.18247*, 2026.

[204] Shiduo Zhang, Zhe Xu, Peiju Liu, Xiaopeng Yu, Yuan Li, Qinghui Gao, Zhaoye Fei, Zhangyue Yin, Zuxuan Wu, Yu-Gang Jiang, et al. Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11142–11152. IEEE, 2025.

[205] Xiaoyu Zhang, Matthew Chang, Pranav Kumar, and Saurabh Gupta. Diffusion meets dagger: Supercharging eye-in-hand imitation learning. *arXiv preprint arXiv:2402.17768*, 2024.

[206] Yuhong Zhang, Zihan Gao, Shengpeng Li, Ling-Hao Chen, Kaisheng Liu, Runqing Cheng, Xiao Lin, Junjia Liu, Zhuoheng Li, Jingyi Feng, et al. Robowheel: A data engine from real-world human demonstrations for cross-embodiment robotic learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6664–6674, 2026.

[207] Zongzheng Zhang, Chenghao Yue, Haobo Xu, Minwen Liao, Xianglin Qi, Huan-ang Gao, Ziwei Wang, and Hao Zhao. Robochemist: Long-horizon and safety-compliant robotic chemical experimentation. *arXiv preprint arXiv:2509.08820*, 2025.

[208] Enyu Zhao, Vedant Raval, Hejia Zhang, Jiageng Mao, Zeyu Shangguan, Stefanos Nikolaidis, Yue Wang, and Daniel Seita. Manipbench: Benchmarking vision-language models for low-level robot manipulation. *arXiv preprint arXiv:2505.09698*, 2025.

[209] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[210] Zhaxizhuom Zhaxizhuoma, Kehui Liu, Chuyue Guan, Zhongjie Jia, Ziniu Wu, Xin Liu, Tianyu Wang, Shuai Liang, Pengan Chen, Pingrui Zhang, et al. Fastumi: A scalable and hardware-independent universal manipulation interface with dataset. In *Conference on Robot Learning*, pages 3069–3093. PMLR, 2025.

[211] Pengfei Zhou, Liliang Chen, Shengcong Chen, Di Chen, Wenzhi Zhao, Rongjun Jin, Guanghui Ren, and Jianlan Luo. Act2goal: From world model to general goal-conditioned policy. *arXiv preprint arXiv:2512.23541*, 2025.

[212] Zhiyuan Zhou, Pranav Atreya, Abraham Lee, Homer Walke, Oier Mees, and Sergey Levine. Autonomous improvement of instruction following skills via foundation models. *arXiv preprint arXiv:2407.20635*, 2024.

[213] Zhiyuan Zhou, Pranav Atreya, You Liang Tan, Karl Pertsch, and Sergey Levine. Autoeval: Autonomous evaluation of generalist robot manipulation policies in the real world. *arXiv preprint arXiv:2503.24278*, 2025.

[214] Adam Zweiger, Jyo Pari, Han Guo, Yoon Kim, and Pulkit Agrawal. Self-adapting language models. *Advances in Neural Information Processing Systems*, 38:74084–74115, 2026.